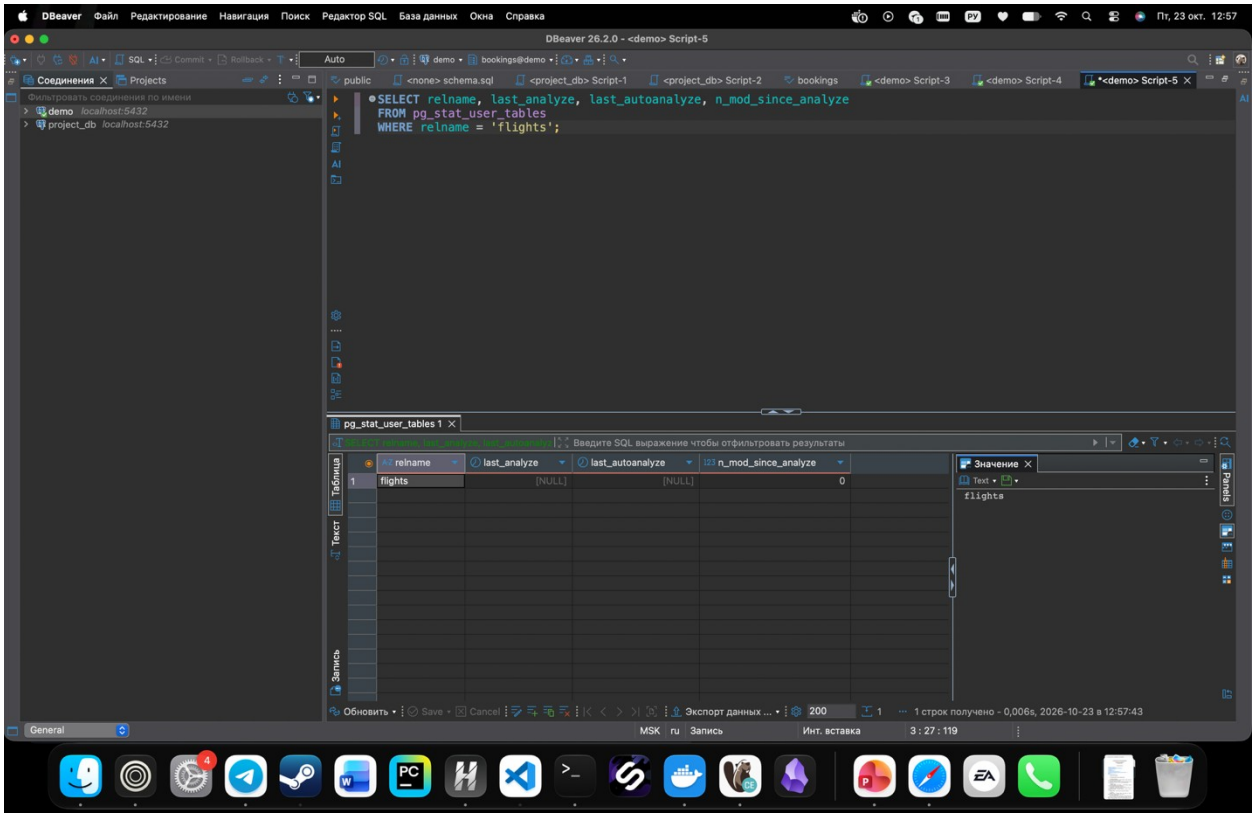
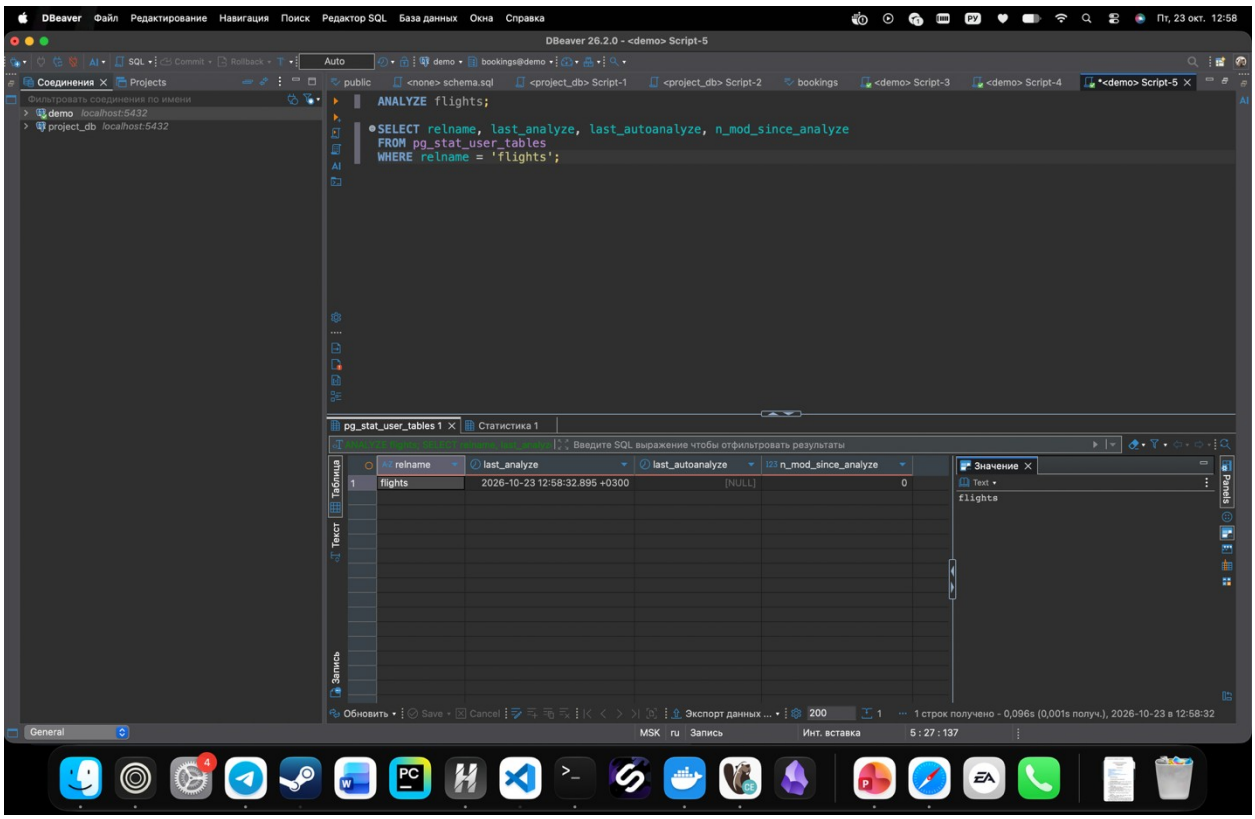
ДЗ 7 БД

Выполнил Чесноков Данила

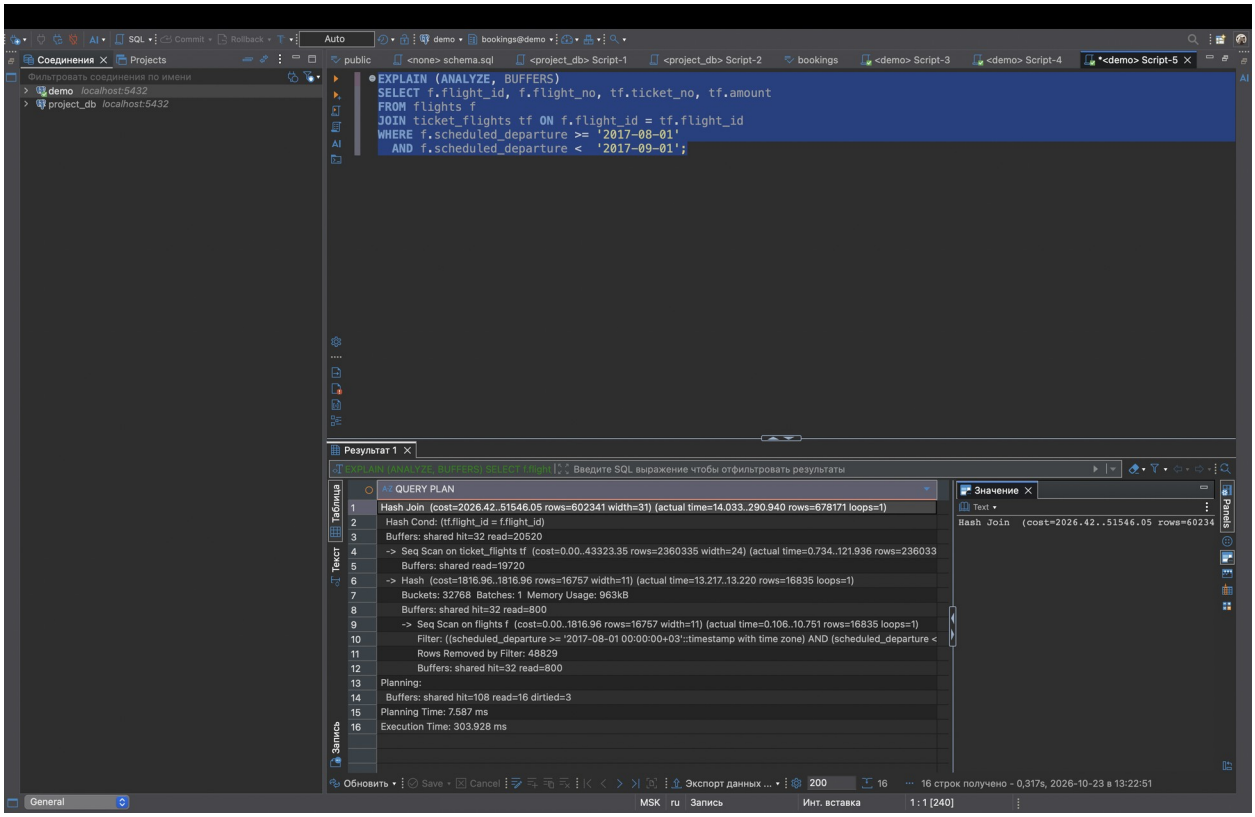
1. Pre-flight Checklist

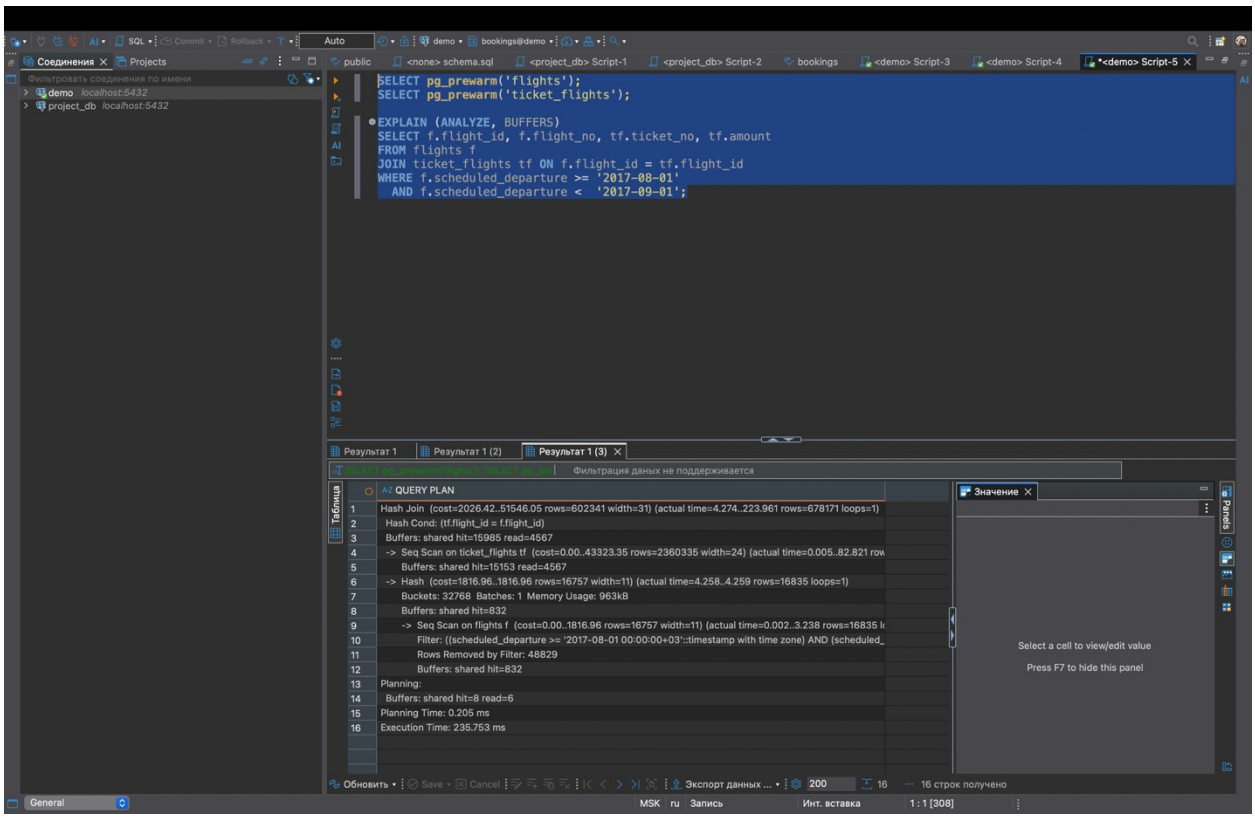




О том, что статистика устарела, судим по n_mod_since_analyze (число изменённых строк с последнего анализа; порог 10% от размера таблицы), а также
по last_analyze и last_autoanalyze (время последнего ручного и автоматического анализа). До выполнения запроса оба поля с датами были NULL, то есть статистика для flights ещё ни разу не собиралась, при этом n_mod_since_analyze = 0. Поэтому был выполнен ANALYZE flights;. После него last_analyze получил актуальную метку времени (2026-10-23 12:58:32), last_autoanalyze остался NULL (он обновляется только автоматическим анализом), а n_mod_since_analyze остался равным 0, то есть планировщик работает со свежей статистикой.

2. Анализ кэша и прогрев

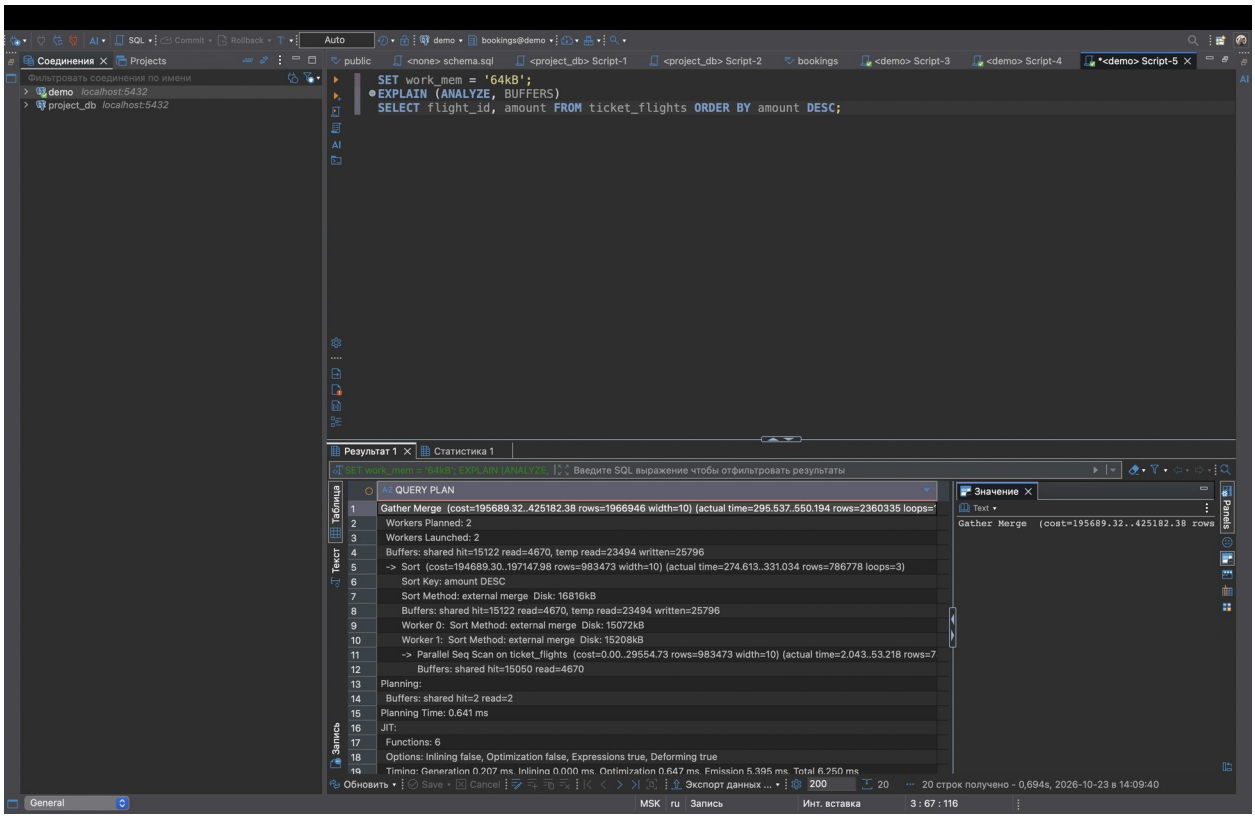


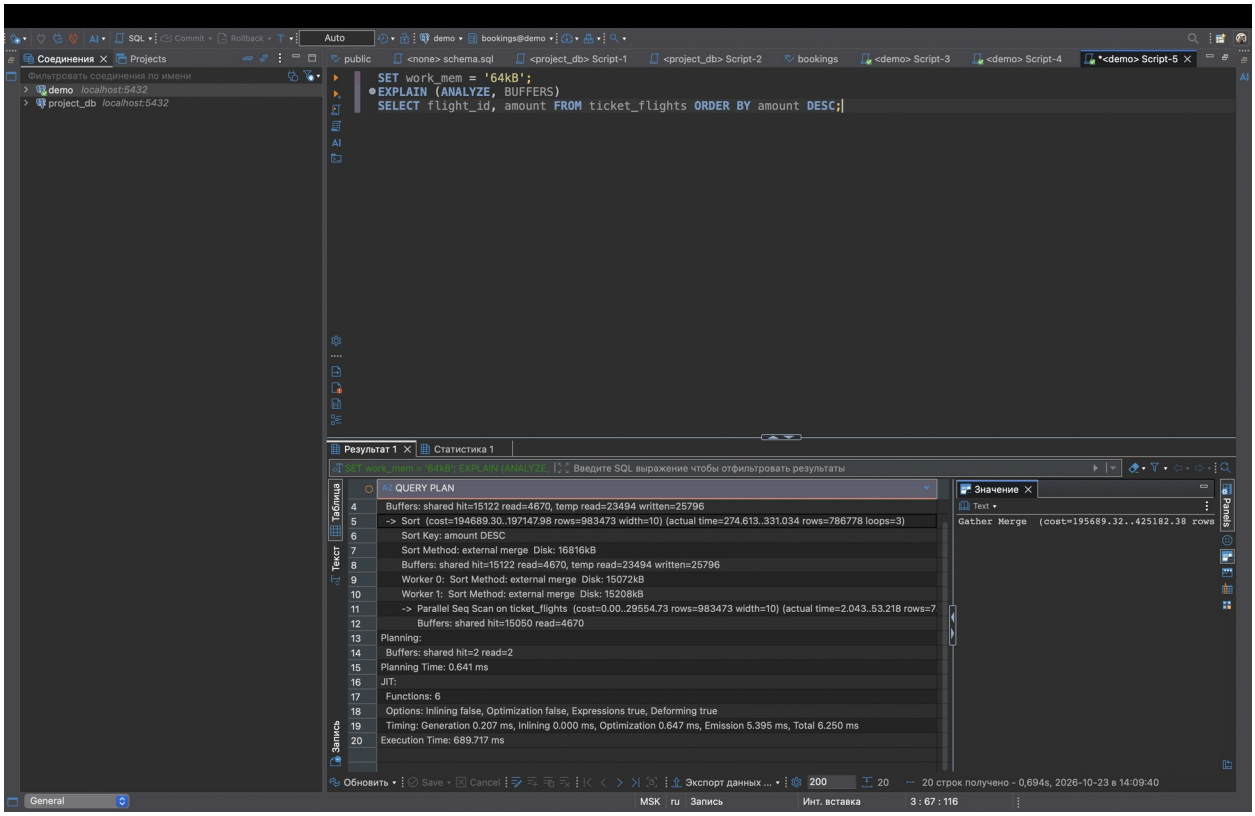


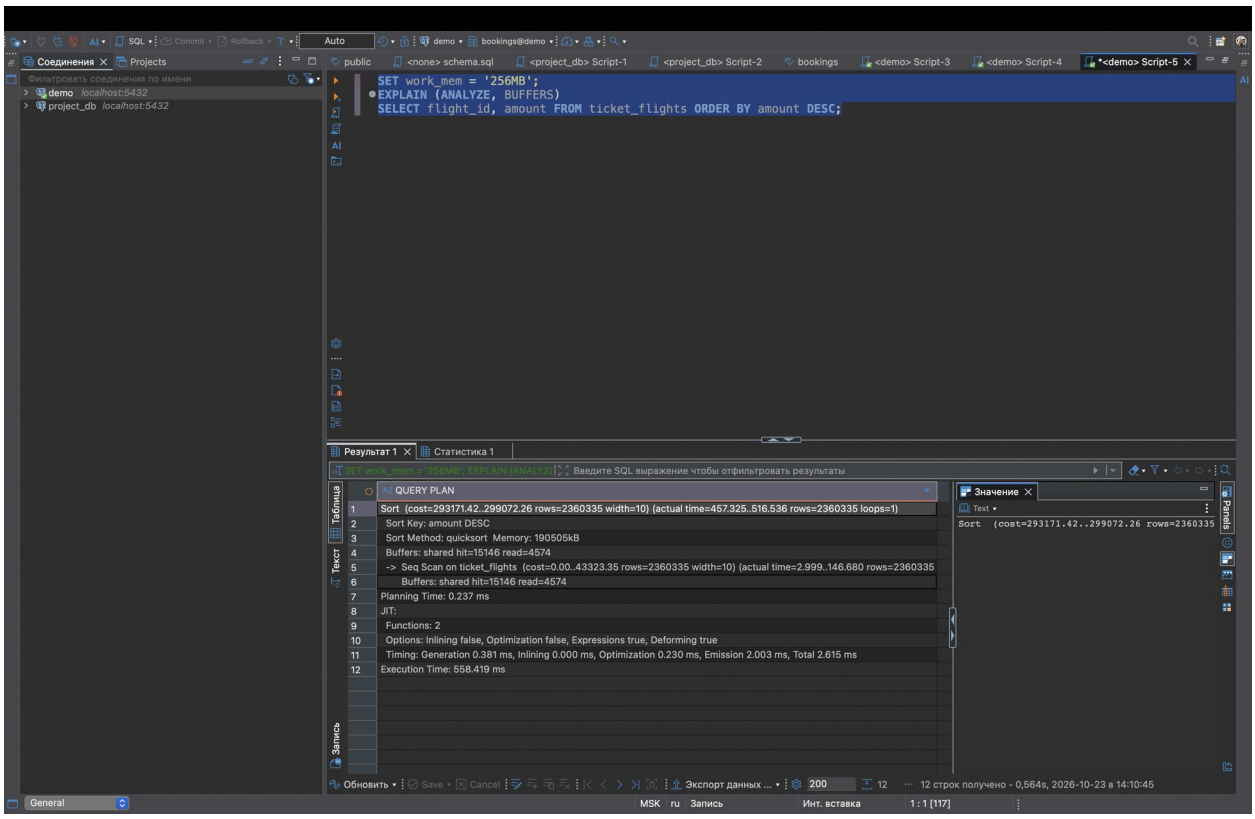
Как pg_prewarm повлиял на план и время. До прогрева данные читались вне shared_buffers (shared read=20520), после pg_prewarm страницы flights целиком оказались в кэше (shared hit=832, read нет), а у ticket_flights загрузилась только часть (hit=15153, read=4567), так как таблица не помещается в shared_buffers (128 МБ). План (Hash Join + Seq Scan) не изменился, а время снизилось с 303,9 до 235,8 мс (~22%): страницы теперь берутся из памяти. Выигрыш умеренный, потому что часть данных всё ещё читается, а сам запрос в основном нагружает процессор (хеш-соединение).

Где видно изменение счётчиков. shared read → shared hit видно в строках 3, 5, 8, 12, а также в строке 14, где read уменьшился с 16 до 6.

3. Исследование `work_mem`





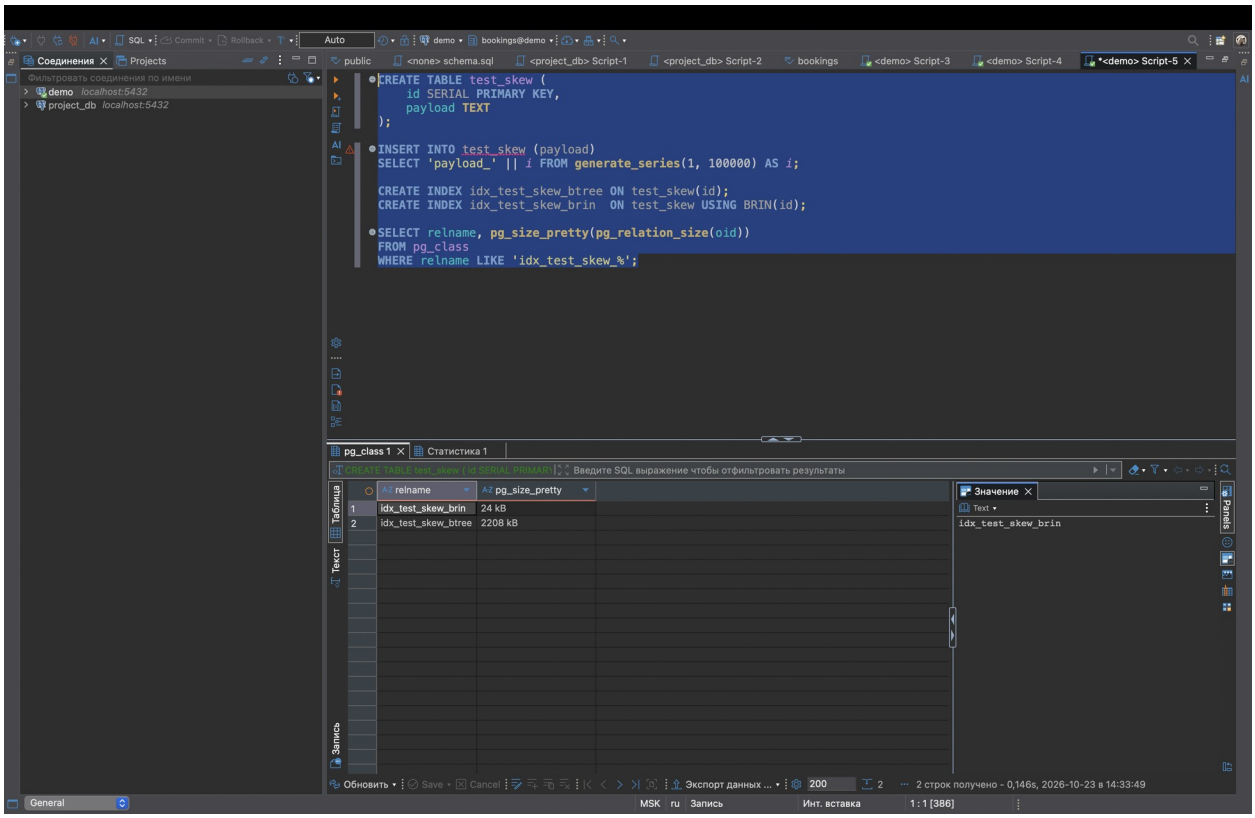


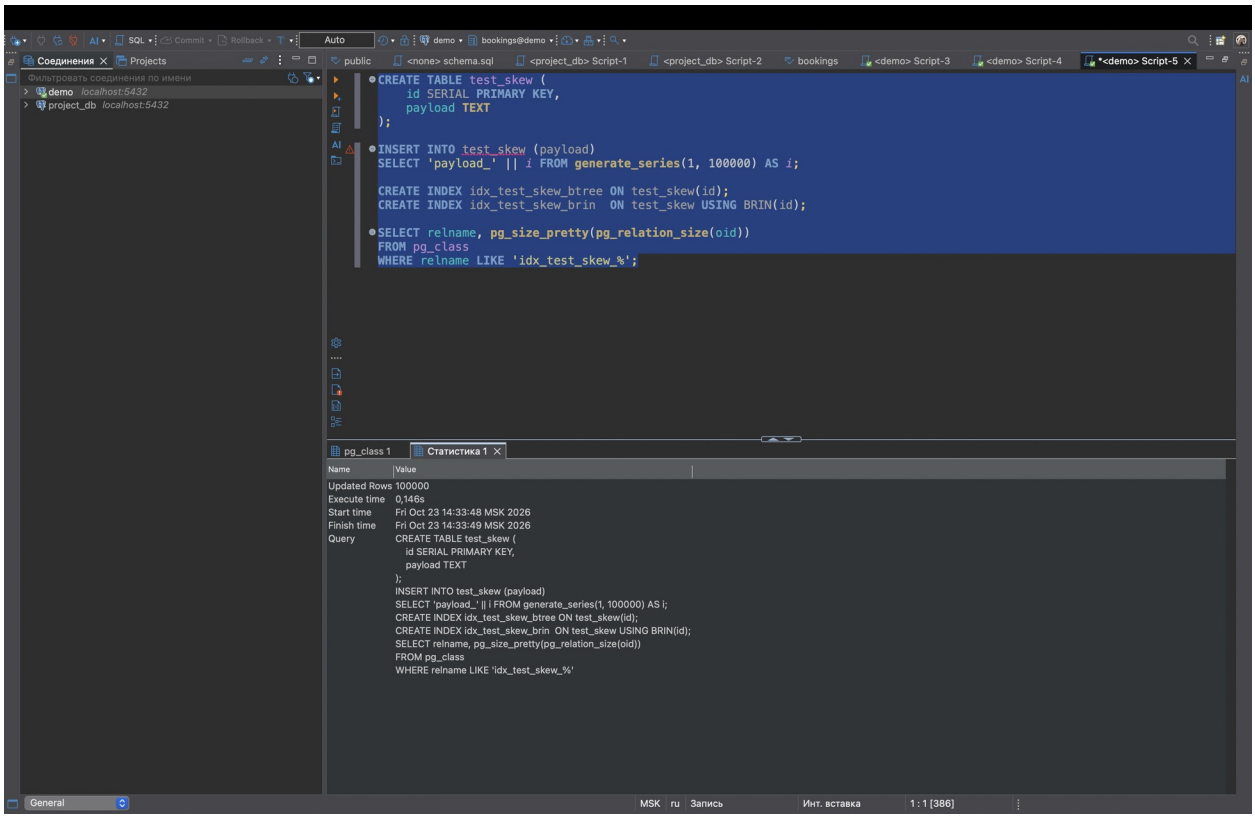
Почему исчезла метрика Disk. Для ORDER BY нужно отсортировать 2,36 млн строк (около 190 МБ). При work_mem = 64kB они не помещаются в память, поэтому PostgreSQL сортирует их частями, сбрасывает во временные файлы на диск и затем читает обратно для слияния (external merge). При work_mem = 256MB всё помещается в память, сортировка идёт целиком в ней (quicksort), временные файлы не нужны, и Disk пропадает.

Влияние на скорость. Время уменьшилось с 689,7 → 568,4 мс, так как исчезли запись и повторное чтение около 200 МБ временных данных. Также изменился план: вместо параллельного Gather Merge выбран обычный Sort.

Почему есть обращения к диску, если таблица в shared_buffers. shared_buffers это общий кэш страниц таблиц и индексов, а work_mem это приватная память отдельной операции для промежуточных результатов (отсортированных данных, хеш-таблиц). Эти результаты не являются страницами таблицы и в shared_buffers не хранятся, поэтому при нехватке work_mem они уходят во временные файлы на диск. Это позволяет сортировать данные любого объёма при ограниченной памяти, но замедляет выполнение.

4. Сравнение индексов (синтетический кейс)





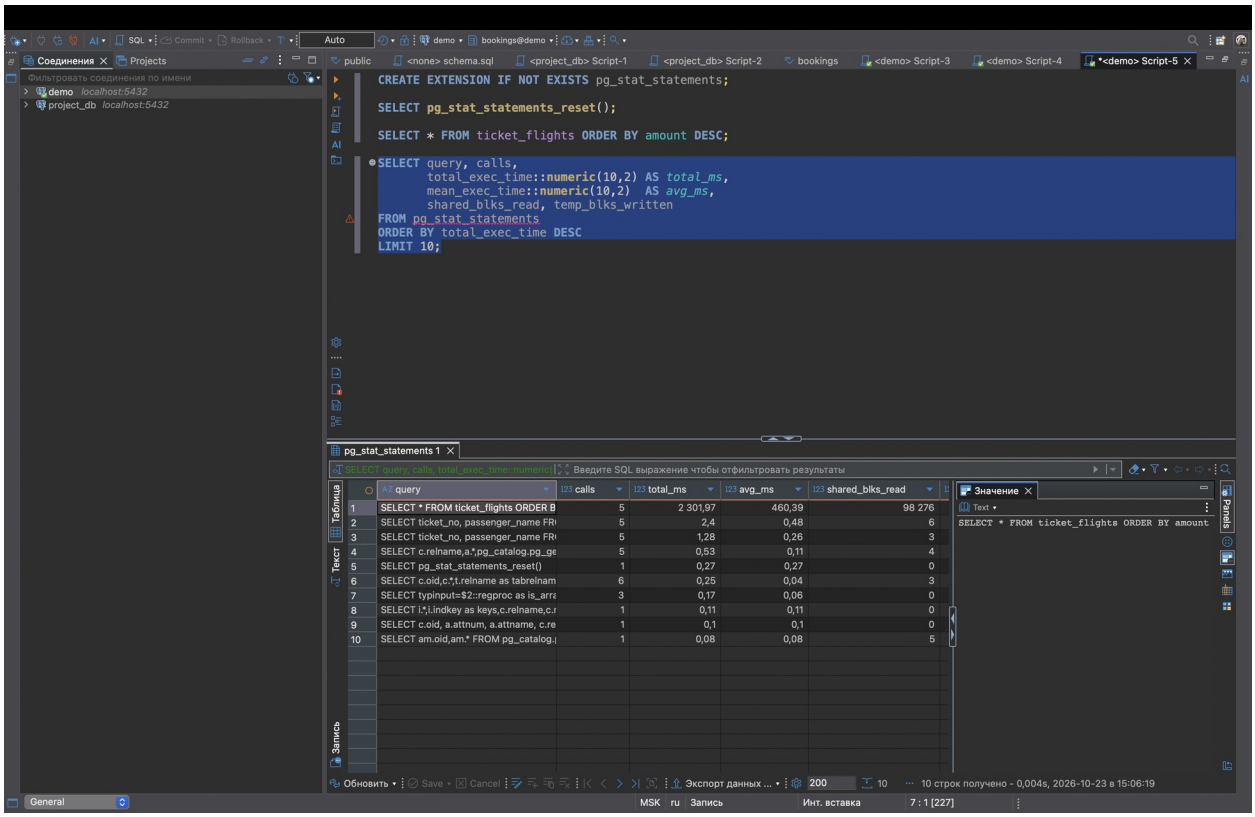
Почему BRIN занимает меньше места. B-Tree хранит отдельную запись для каждой строки таблицы (значение ключа и указатель на строку, TID), поэтому для 100 000 строк его размер составляет 2208 кБ и растёт пропорционально числу строк. BRIN делит таблицу на диапазоны по 128 страниц и хранит для каждого диапазона только минимальное и

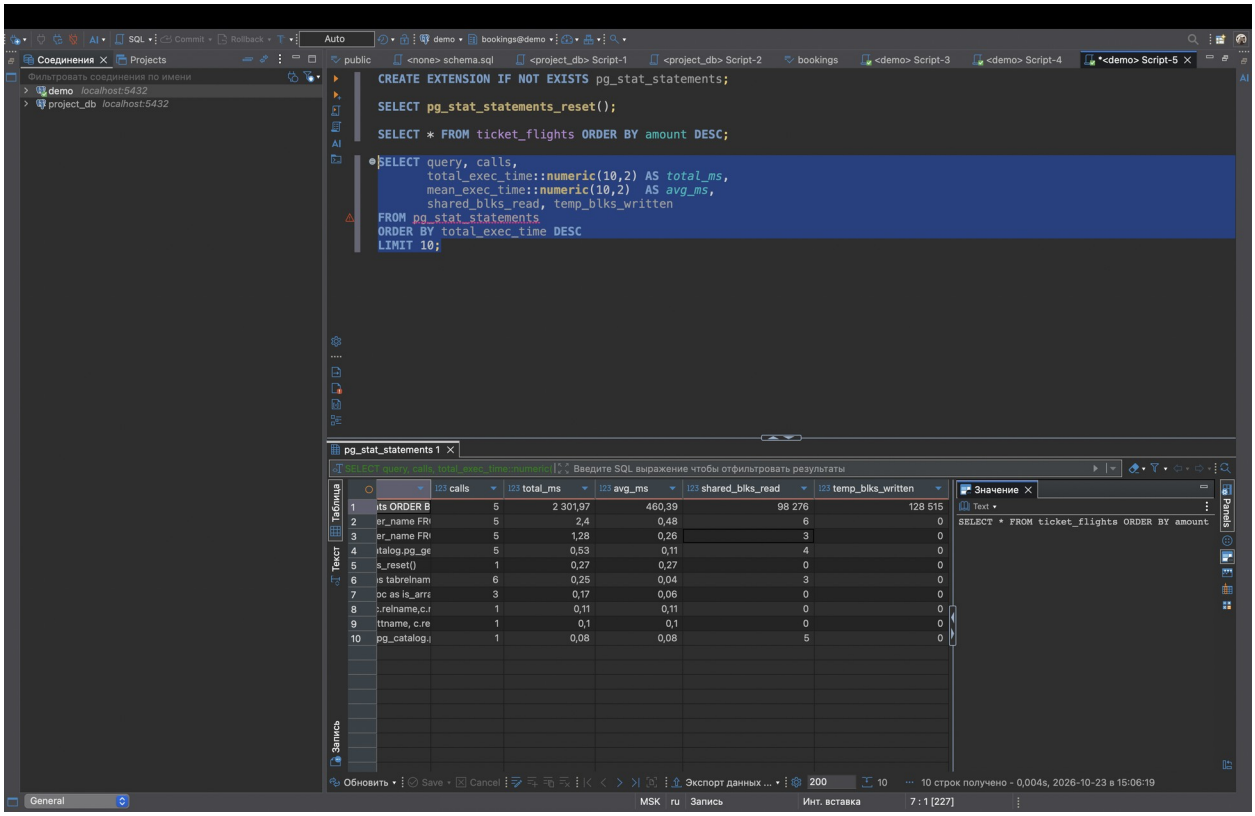
максимальное значение id, без ссылок на отдельные строки. Диапазонов всего около пяти, поэтому индекс занимает 24 кБ (в основном служебные страницы), то есть примерно в 90 раз меньше. Платой за компактность является точность: BRIN лишь отбрасывает диапазоны, где искомого значения быть не может, поэтому он эффективен, когда значения столбца совпадают с физическим порядком строк. Для id из SERIAL это выполняется.

Когда использовать B-Tree, а когда BRIN. B-Tree хранит точную ссылку на каждую строку, поэтому он подходит для OLTP-сценариев с поиском по ID: быстро находит одну строку или малую выборку и работает независимо от физического порядка данных, но занимает много места и замедляет запись. BRIN хранит только минимум и максимум по диапазонам страниц, поэтому он подходит для очень больших таблиц с дописываемыми данными, например логов, где значение (время или ID) растёт вместе с физическим порядком строк, а запросы читают диапазон значений. Он очень компактен и почти не замедляет вставку, но при перемешанных данных отсекает мало диапазонов и неэффективен.

Как помогают fillfactor и партиционирование. fillfactor оставляет на страницах резерв свободного места. В OLTP-сценариях с B-Tree и частыми обновлениями (fillfactor 80-90) это позволяет делать HOT-обновления без изменения индексов и реже расщеплять страницы индекса, что снижает раздувание. Для логов с BRIN резерв не нужен: данные только дописываются, поэтому fillfactor = 100 даёт максимально плотное хранение. Партиционирование (например, по дате) делит таблицу на части: запросы читают только нужные партиции (partition pruning), индексы на каждой партиции получаются меньше и лучше помещаются в кэш, а BRIN внутри партиции работает на упорядоченных данных. Устаревшие данные удаляются целой партицией через DROP/DETACH без DELETE и последующей очистки VACUUM.

5.Диагностика pg_stat_statements





Эффективность запроса. Запрос SELECT * FROM ticket_flights ORDER BY amount DESC оказался самым тяжёлым: 5 вызовов, суммарно 2301,97 мс, в среднем 460,39 мс,

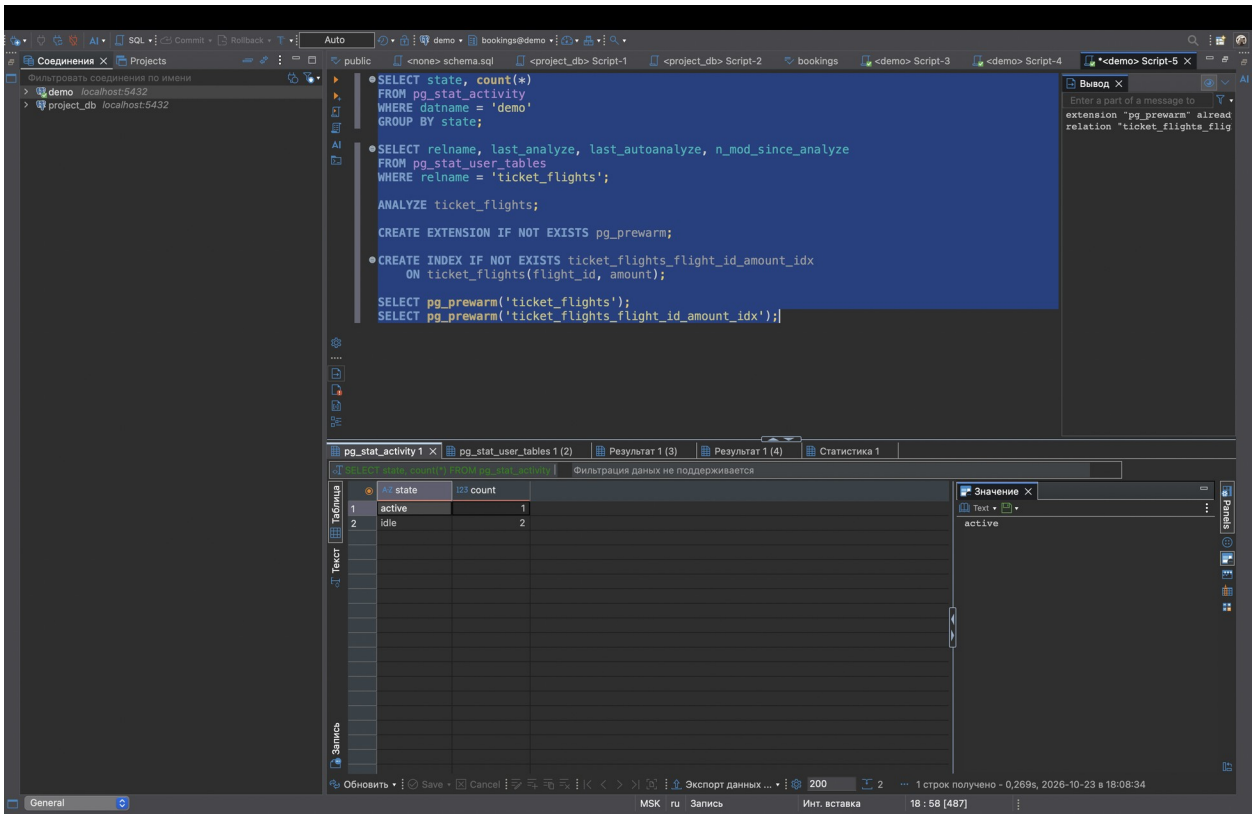
остальные запросы в топе занимают единицы миллисекунд. Он неэффективен по двум причинам. Во-первых, temp_blks_written = 128515 (около 25,7 тыс. блоков на вызов, примерно 200 МБ): сортировка 2,36 млн строк не помещается в work_mem и сбрасывается во временные файлы на диск. Во-вторых, shared_blks_read = 98276 (около 19,7 тыс. блоков на вызов, то есть вся таблица): данные каждый раз читаются с диска, потому что таблица не помещается в shared_buffers. Ускорить запрос можно так: ограничить выборку (WHERE, LIMIT), выбирать только нужные столбцы вместо *, создать индекс по amount, чтобы не сортировать заново, или увеличить work_mem для сессии.

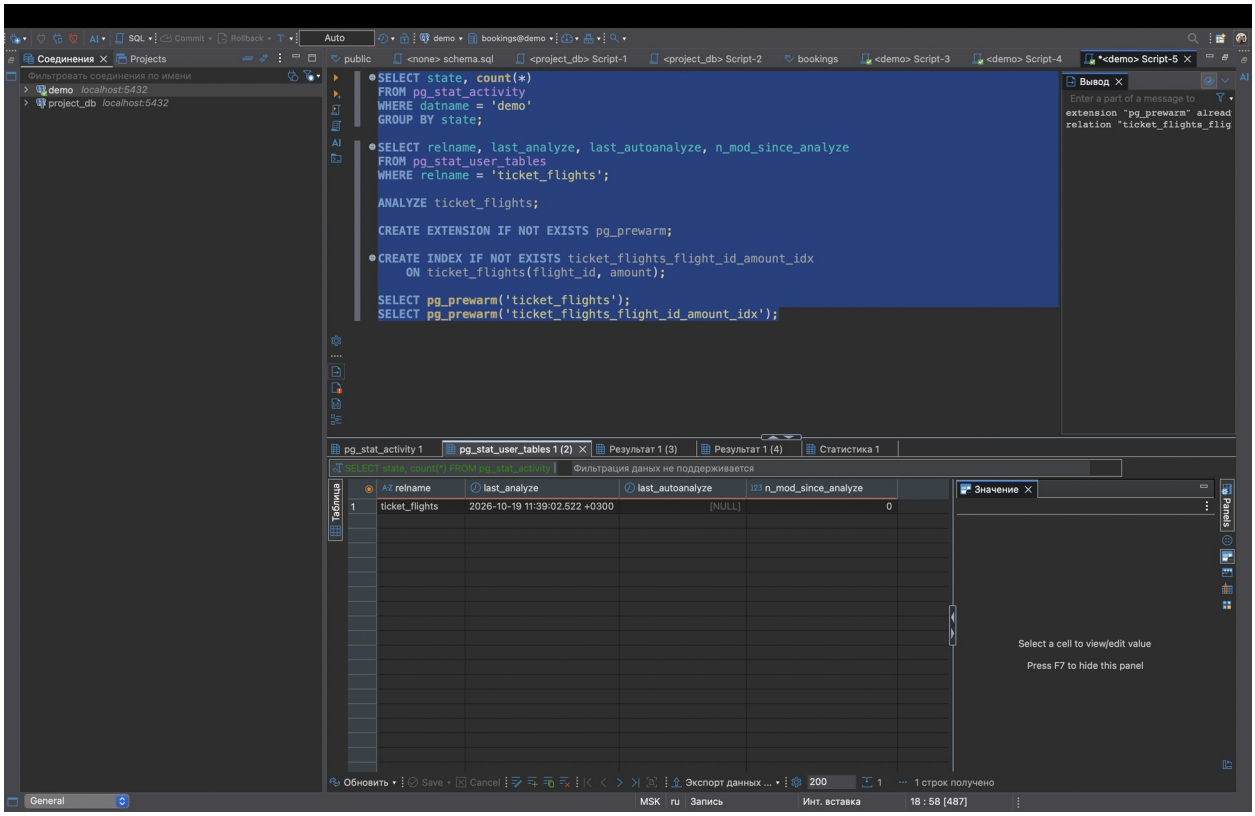
6. Несделанные задачи с практики

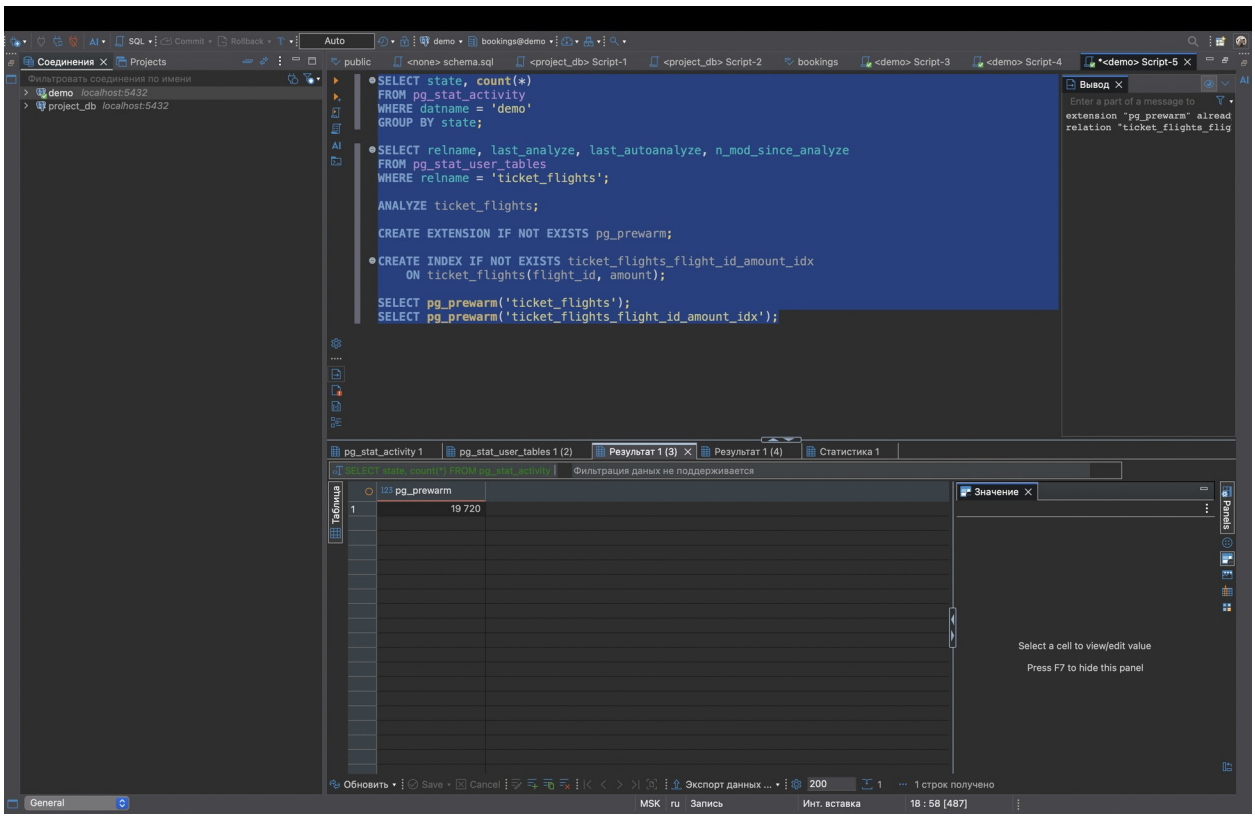
Раздел 1. Подготовка окружения

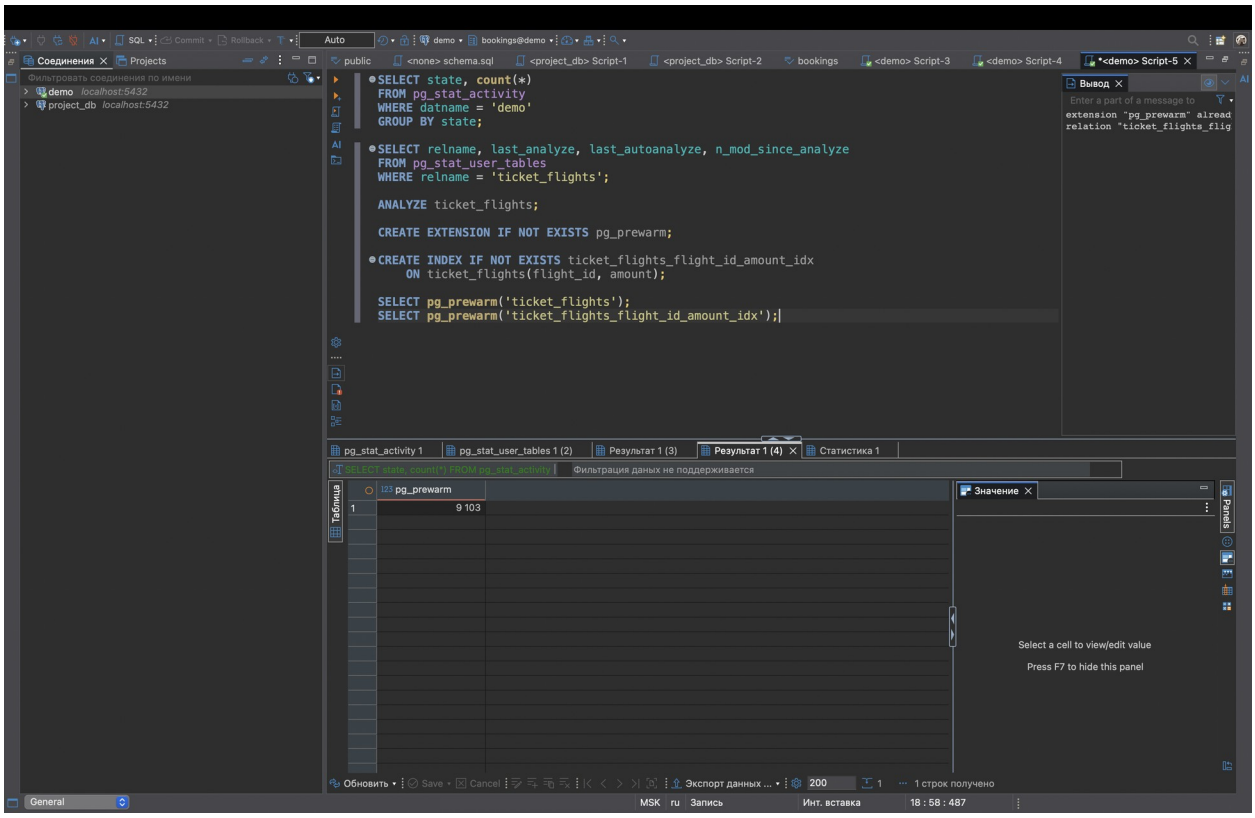
Подготовка окружения. Первая команда переходит в корневую папку с материалами курса (если переменная `UNIVERSITY_ROOT` не задана, берётся каталог `university`). Вторая сначала пытается запустить существующий контейнер `pg16_check`, а если его нет, создаёт новый из образа `postgres:16` в фоне (`-d`) с паролем `postgres` и пробросом порта 5432, к которому подключается DBeaver. Благодаря конструкции `||` команду можно выполнять повторно без ошибок. Третья команда запускает скрипт `seminar7_setup.sh`, который готовит базу `demo` с данными для заданий, поэтому стенд у всех одинаковый и воспроизводимый. Четвёртая открывает внутри контейнера интерактивный клиент `psql`, подключённый к `demo` под пользователем `postgres`, и подтверждает, что контейнер работает, а база создана и принимает запросы. Префикс `MSYS_NO_PATHCONV=1` нужен только в Git Bash на Windows: он отключает автоматическую подмену путей, которая могла бы исказить аргументы команды. Основные замеры выполняются в DBeaver, а `psql` используется для проверки стенда и запуска скриптов.

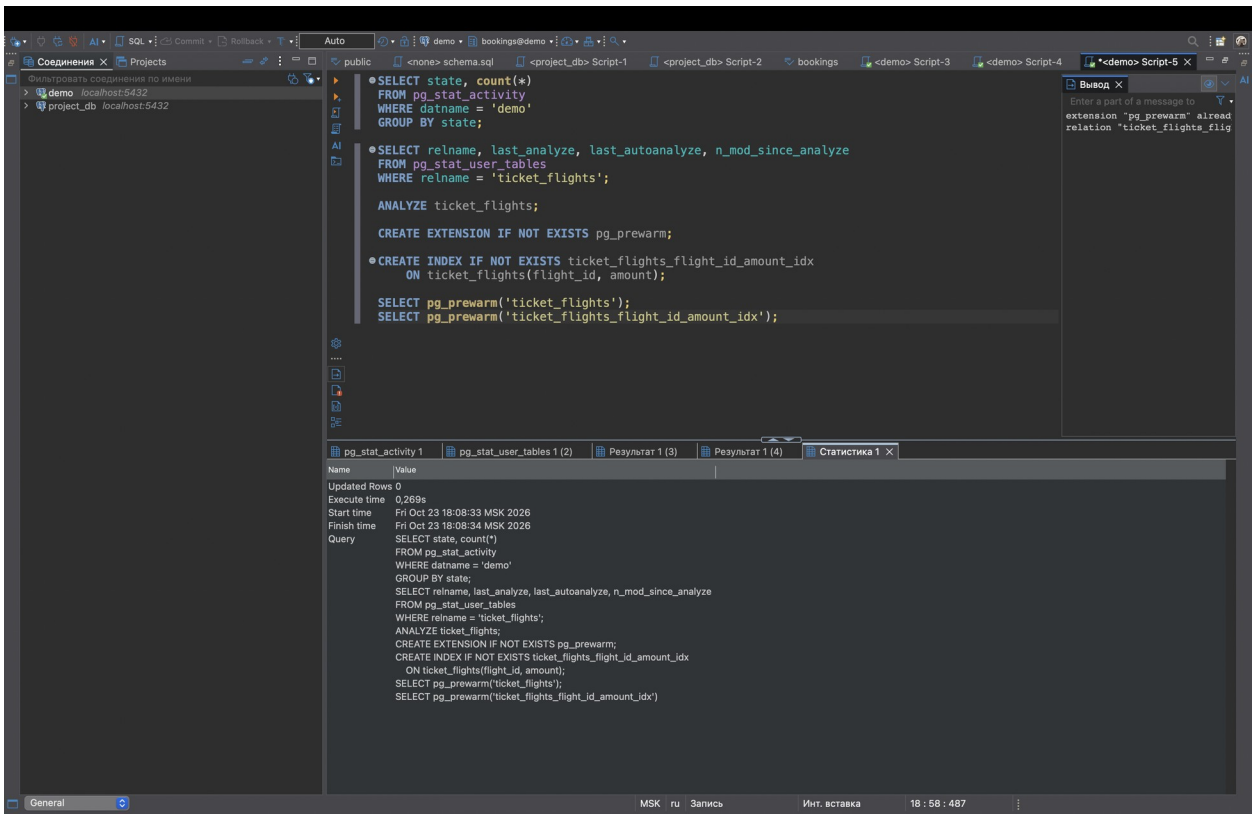
Раздел 3. Практикум 1: Pre-flight Checklist





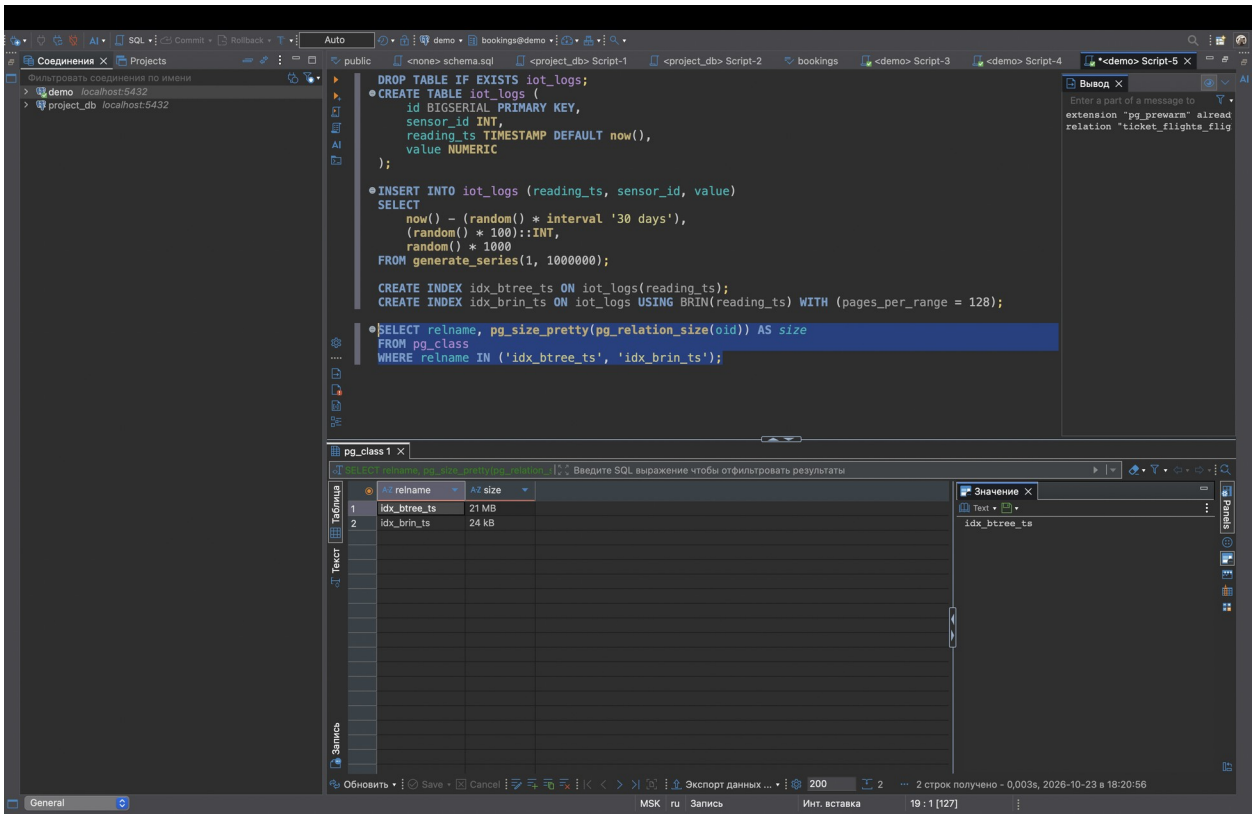

SELECT state, count(*)
FROM pg_stat_activity
WHERE datname = 'demo'
GROUP BY state;
SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
FROM pg_stat_user_tables
WHERE relname = 'ticket_flights';
ANALYZE ticket_flights;
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
ON ticket_flights(flight_id, amount);
SELECT pg_prewarm('ticket_flights');
SELECT pg_prewarm('ticket_flights_flight_id_amount_idx');
pg_prewarm
1
19 720
Select a cell to view/edit value
Press F7 to hide this panel


SELECT state, count(*)
FROM pg_stat_activity
WHERE datname = 'demo'
GROUP BY state;
SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
FROM pg_stat_user_tables
WHERE relname = 'ticket_flights';
ANALYZE ticket_flights;
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
ON ticket_flights(flight_id, amount);
SELECT pg_prewarm('ticket_flights');
SELECT pg_prewarm('ticket_flights_flight_id_amount_idx');
pg_prewarm
1
9 103
Select a cell to view/edit value
Press F7 to hide this panel



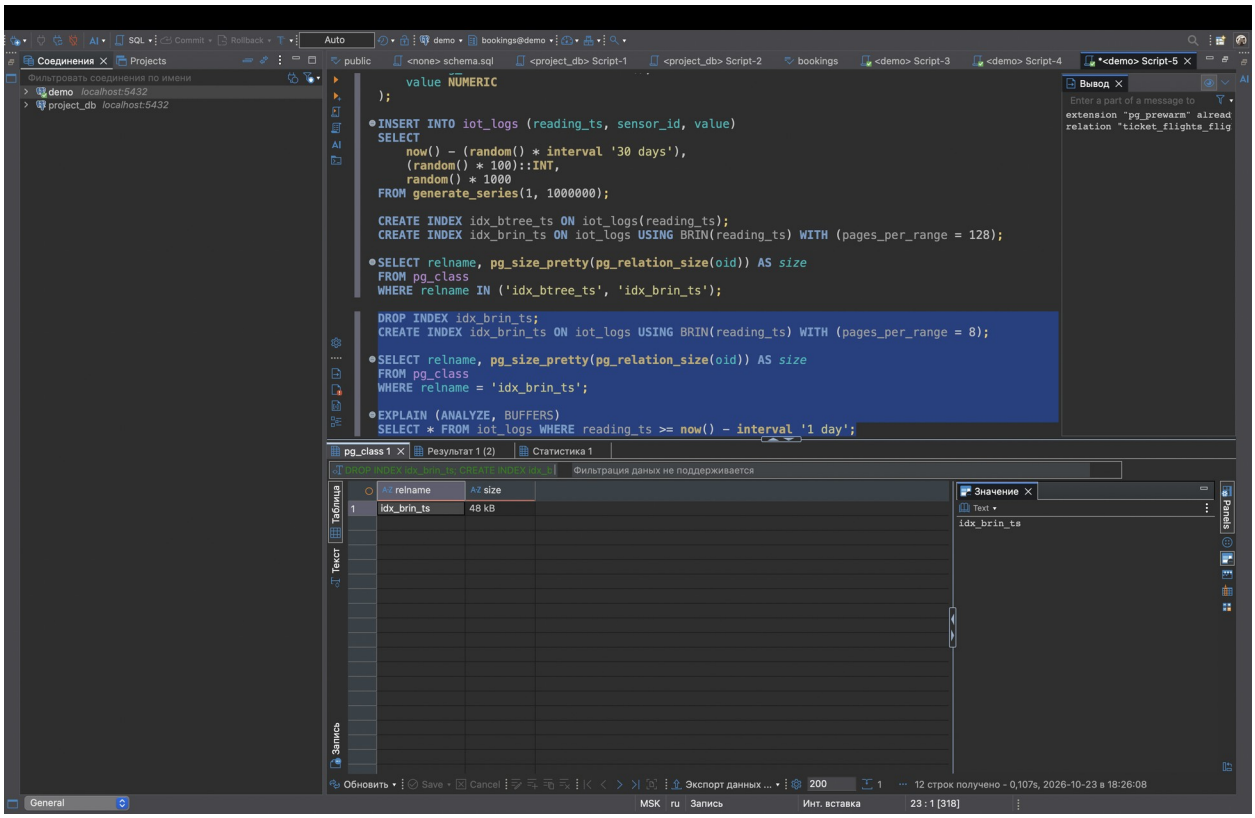
Состояния сессий в pg_stat_activity. Запрос группирует подключения к базе demo по полю state и показывает, что сейчас делает каждая сессия. На скриншоте одна сессия в состоянии active и две в состоянии idle. Состояние active означает, что сессия в этот момент выполняет запрос. Здесь это сам текущий запрос из DBeaver, потому что pg_stat_activity показывает и вызвавшую его сессию. Состояние idle означает, что соединение открыто, но запросов не выполняет и ждёт команды. Это, вероятно, другие вкладки и служебные соединения DBeaver. Состояний idle in transaction (незавершённая транзакция, которая держит блокировки и мешает работе VACUUM) и других активных сессий нет, то есть посторонней нагрузки на базу нет. Это важно проверять перед замерами, потому что EXPLAIN (ANALYZE) измеряет реальное время выполнения, и параллельные запросы конкурируют за процессор, диск и shared_buffers, искажают время и вытесняют из кэша прогретые страницы, а долгие транзакции могут создавать блокировки. Если перед замером в выводе только своя активная сессия и простаивающие соединения, результаты read, hit и Execution Time можно считать сопоставимыми. При этом pg_stat_activity показывает состояние только на момент запроса, а не историю, поэтому проверку лучше повторять непосредственно перед каждой серией замеров.

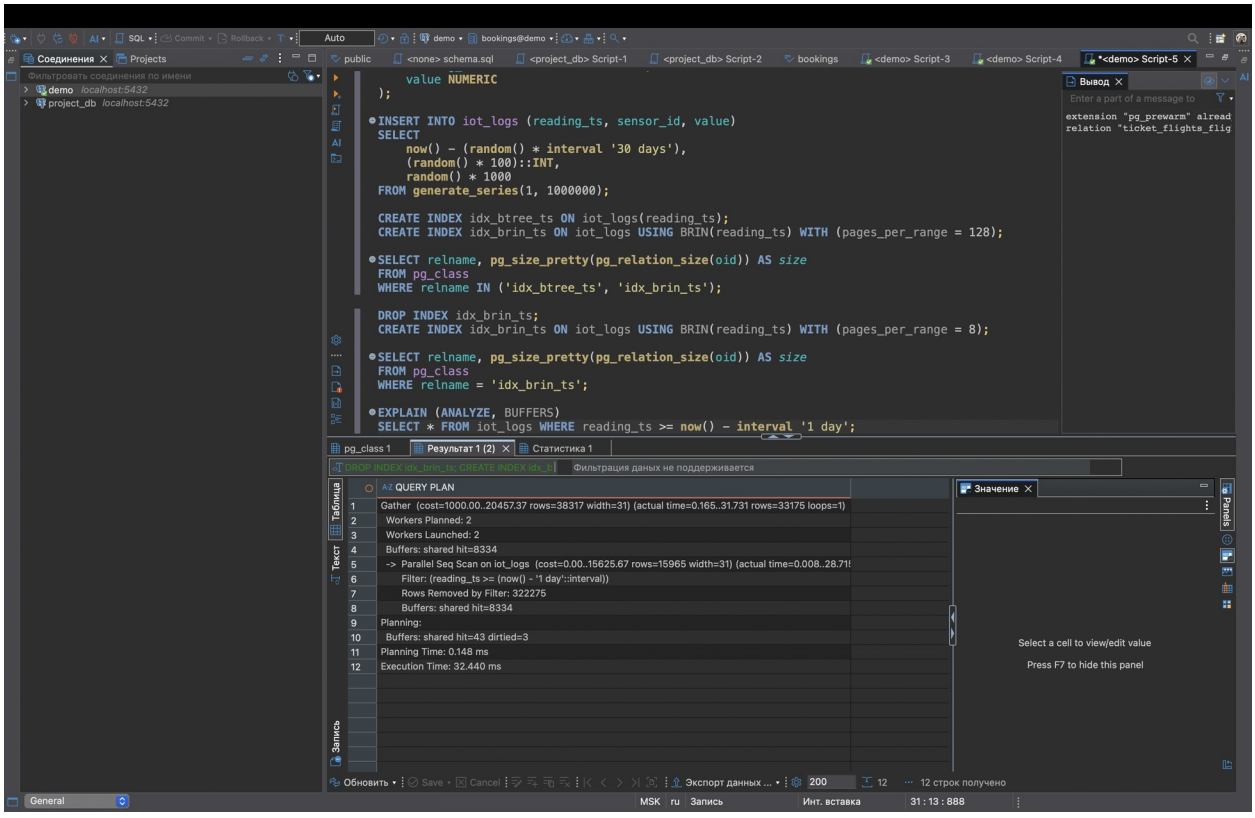
Раздел 4. Практикум 2: Перекос B-Tree, BRIN vs B-Tree

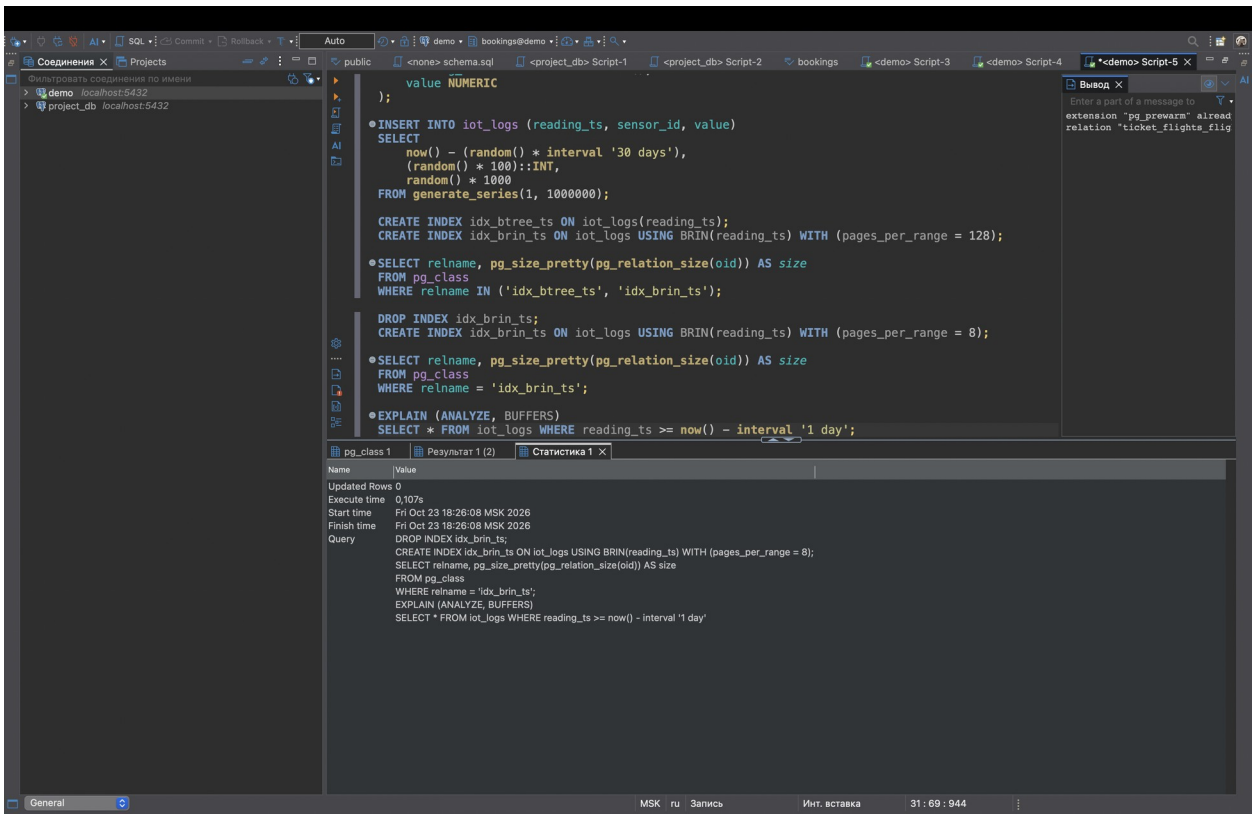


Сравнение размеров индексов. Запрос к pg_class возвращает размеры двух индексов, построенных по одному столбцу reading_ts таблицы iot_logs на 1 млн строк: B-Tree idx_btree_ts занимает 21 МБ, BRIN idx_brin_ts с pages_per_range = 128 занимает 24 КБ, то есть меньше почти в 900 раз. Разница возникает из-за того, что B-Tree хранит в листовых страницах запись для каждой строки таблицы (значение ключа и ссылку на строку), поэтому его размер растёт пропорционально числу строк. BRIN же хранит только минимальное и максимальное значение для каждого диапазона из 128 соседних страниц таблицы, поэтому его размер зависит от числа диапазонов, а не от числа строк, и индекс умещается в несколько страниц. Компактность BRIN имеет свою цену: он не знает, где именно лежит нужная строка, а только в каких диапазонах она может находиться, поэтому после поиска по нему приходится перечитывать все страницы подходящих диапазонов и перепроверять строки. Пользу такой индекс даёт только тогда, когда значения в таблице упорядочены физически, как в журналах, которые пишутся по времени. В нашей таблице reading_ts вставлен в случайном порядке, поэтому по размеру BRIN выигрывает, но фильтровать данные он будет плохо, что видно в плане запроса.

Самостоятельное задание

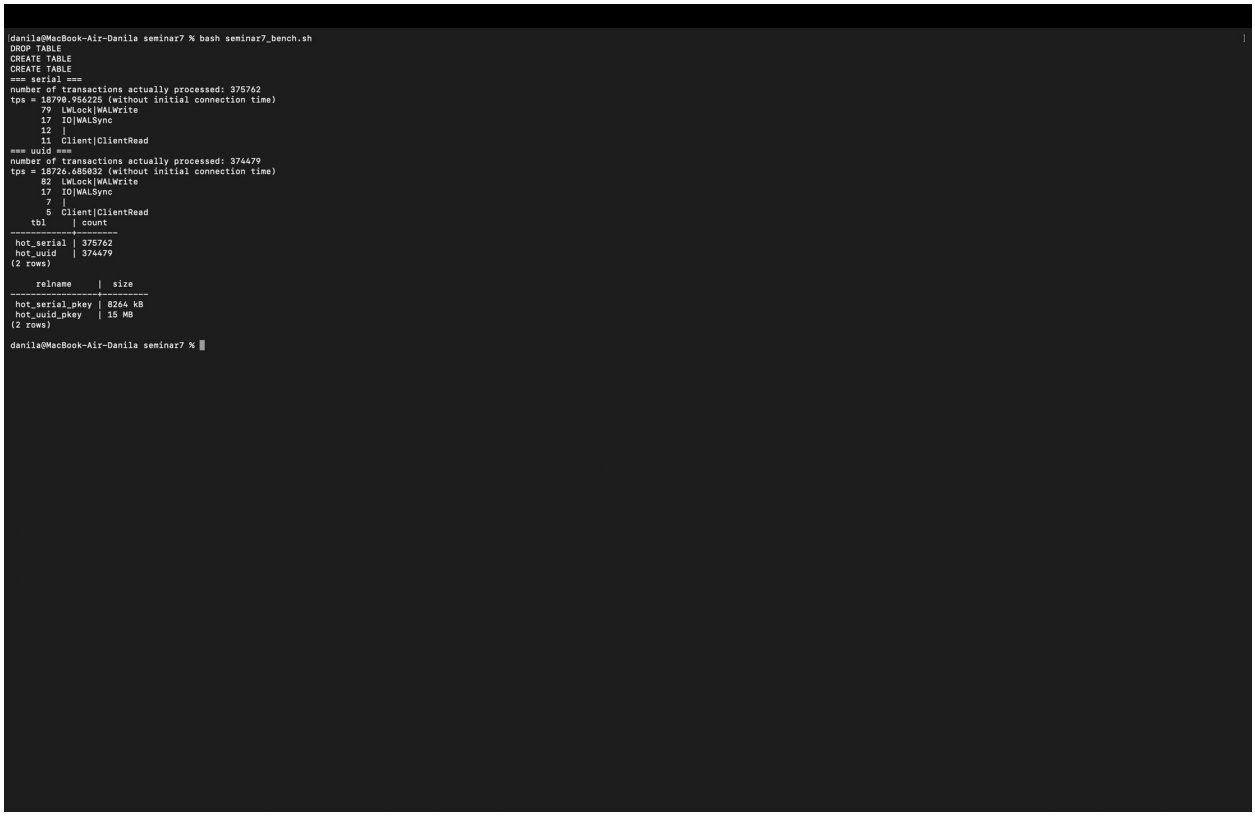






Влияние pages_per_range и план запроса. Параметр pages_per_range задаёт, сколько страниц таблицы описывает одна запись BRIN (min/max). При уменьшении со 128 до 8 диапазонов становится больше, поэтому индекс вырос с 24 КБ до 48 КБ. Зато диапазоны уже, их min/max точнее и могут отсечь больше лишних страниц, а при большом pages_per_range индекс компактнее, но грубее. В плане запроса индексы не использованы: выбран Parallel Seq Scan (строка 5), в каждом процессе отфильтровано по 322 275 строк (строка 7, среднее при loops=3, то есть около 967 тыс. из 1 млн), все 8334 страницы прочитаны из кэша. Причина в том, что reading_ts вставлен в случайном порядке: строки за последние сутки разбросаны по всей таблице, и в каждый диапазон, даже из 8 страниц, попадают и ранние, и поздние даты, поэтому BRIN ничего не отсекает. Размер BRIN определяется pages_per_range, а польза от него зависит от физической упорядоченности данных.

Раздел 6. Практикум 4: Latch Contention и Hot Spot

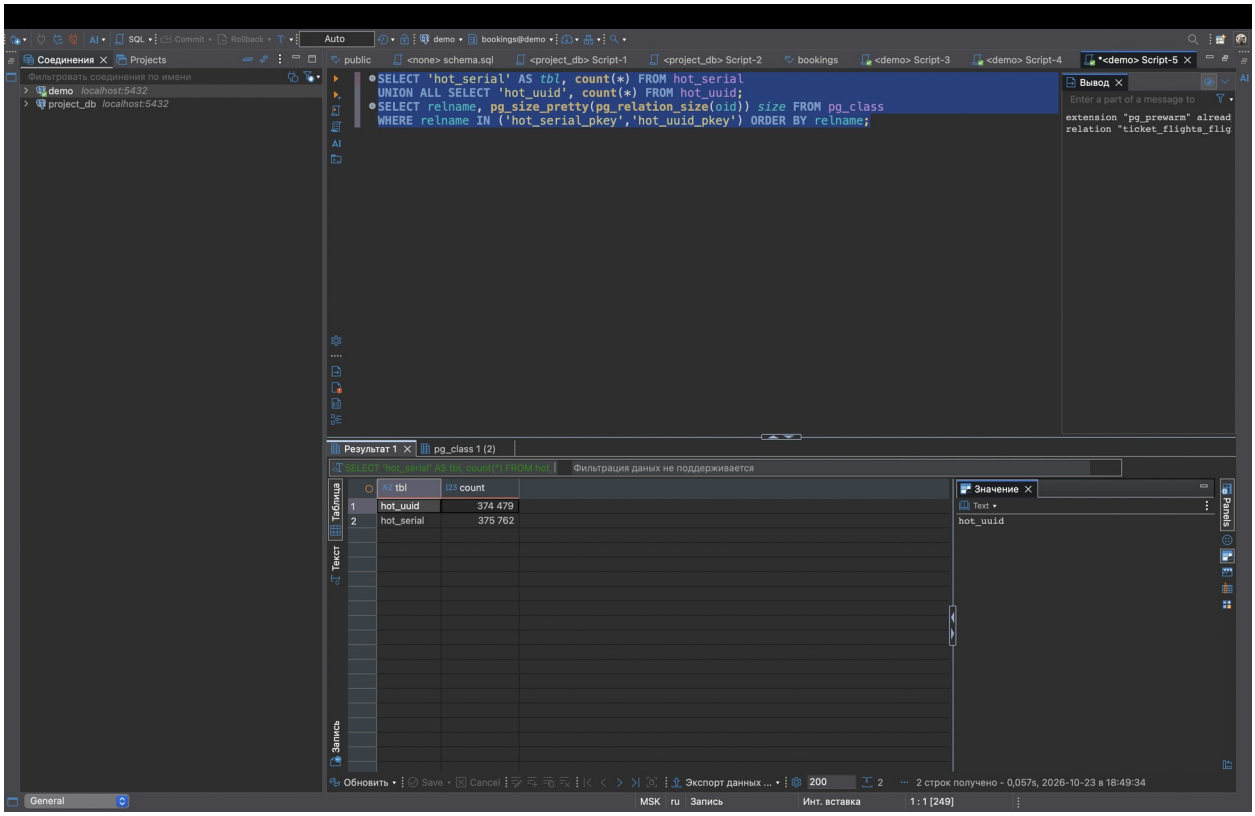


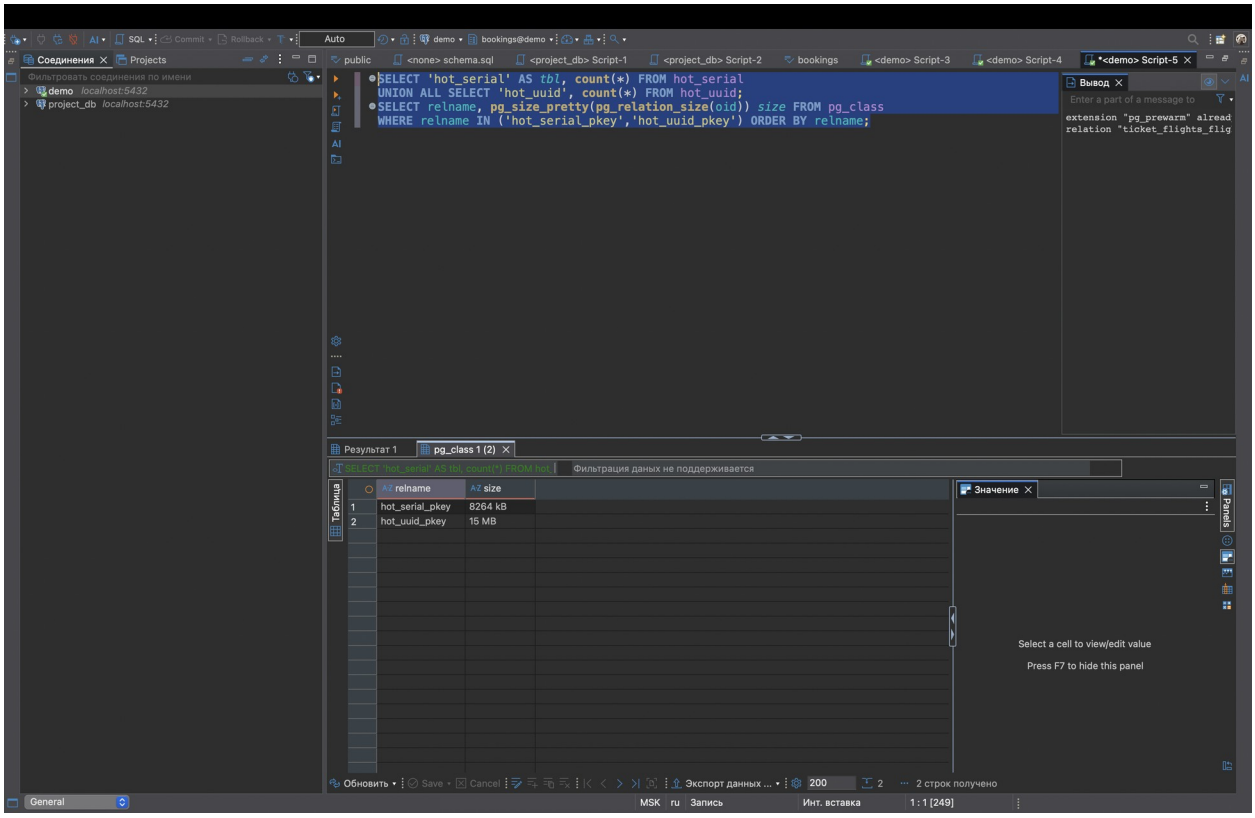
Что делает pgbench. pgbench — штатная утилита PostgreSQL для нагрузочного тестирования: она открывает несколько параллельных соединений (-c 8) в нескольких потоках (-j 4), и каждое соединение в цикле выполняет SQL из файла (-f) в течение заданного времени (-T 20), после чего выдаёт число транзакций и tps. Флаг -n отключает служебный vacuum перед тестом. В этом практикуме одна транзакция равна одной вставке, поэтому pgbench создаёт конкурентную нагрузку на вставки в первичный ключ.

Сравнение wait events для hot_serial и hot_uuid. В обоих режимах топ ожиданий практически одинаков: лидирует LWLock:WALWrite, затем IO:WALSync и Client:ClientRead, а tps и число обработанных транзакций различаются меньше чем на 1%. Конкуренции за одну страницу индекса (LWLock:BufferContent) у hot_serial не видно. Каждая вставка здесь является отдельной короткой транзакцией, поэтому на 8 клиентах узким местом становится запись и сброс WAL при коммите. Эта очередь общая для обоих вариантов и перекрывает эффект hot spot, поэтому выигрыша в скорости от UUID на этом стенде нет.

Почему UUID-PK снимает hot spot и чем это оплачивается. При BIGSERIAL значения монотонно растут, и все вставки попадают в самый правый лист B-Tree, из-за чего при высокой конкуренции процессы ждут друг друга на одной странице. Случайный UUID распределяет вставки по всему индексу, и конкуренция за правый край исчезает. Цена этого видна по размерам: hot_uuid_pkey почти вдвое больше hot_serial_pkey при том же числе строк. Вставки в случайные места приводят к page splits и неплотному заполнению страниц, ухудшают локальность кэша, а сам ключ занимает 16 байт вместо 8. Поэтому

UUID оправдан при реальной конкуренции за правый край индекса, а на нашей нагрузке он дал только рост индекса.



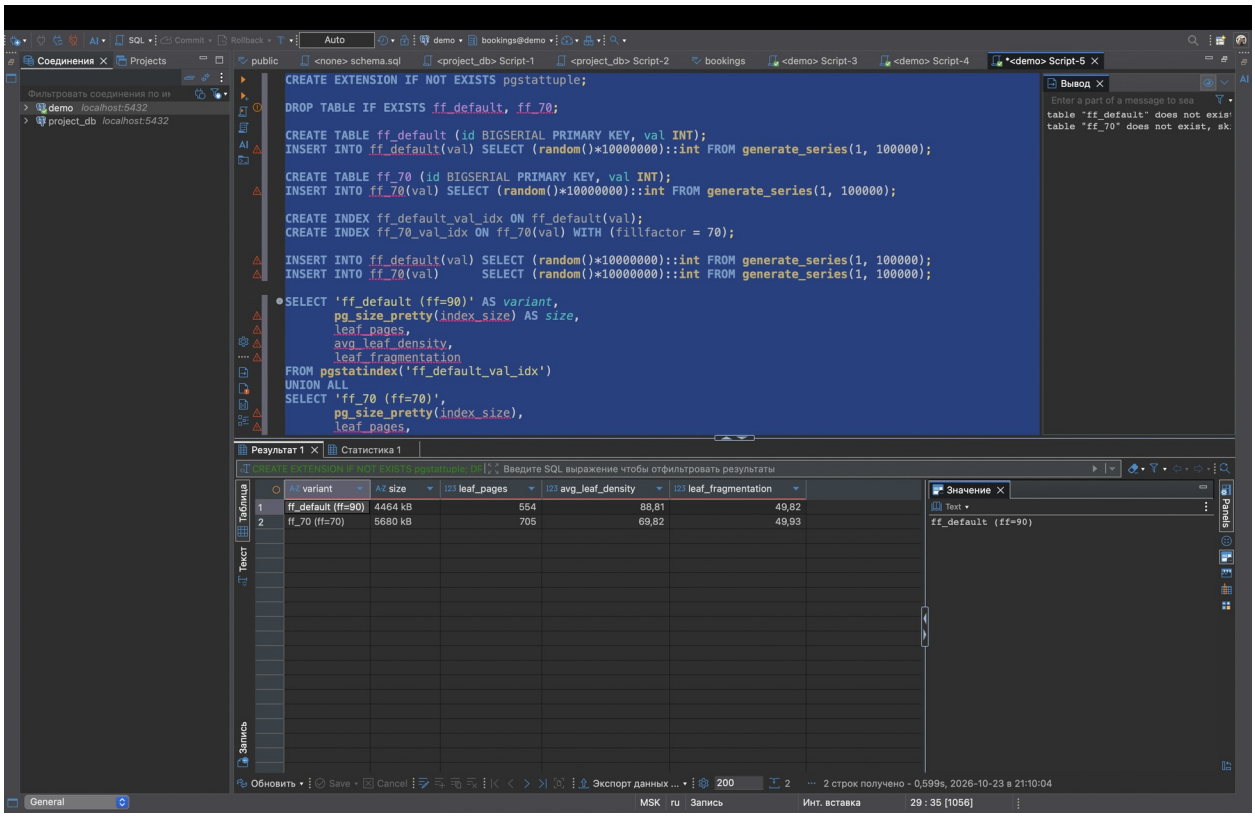


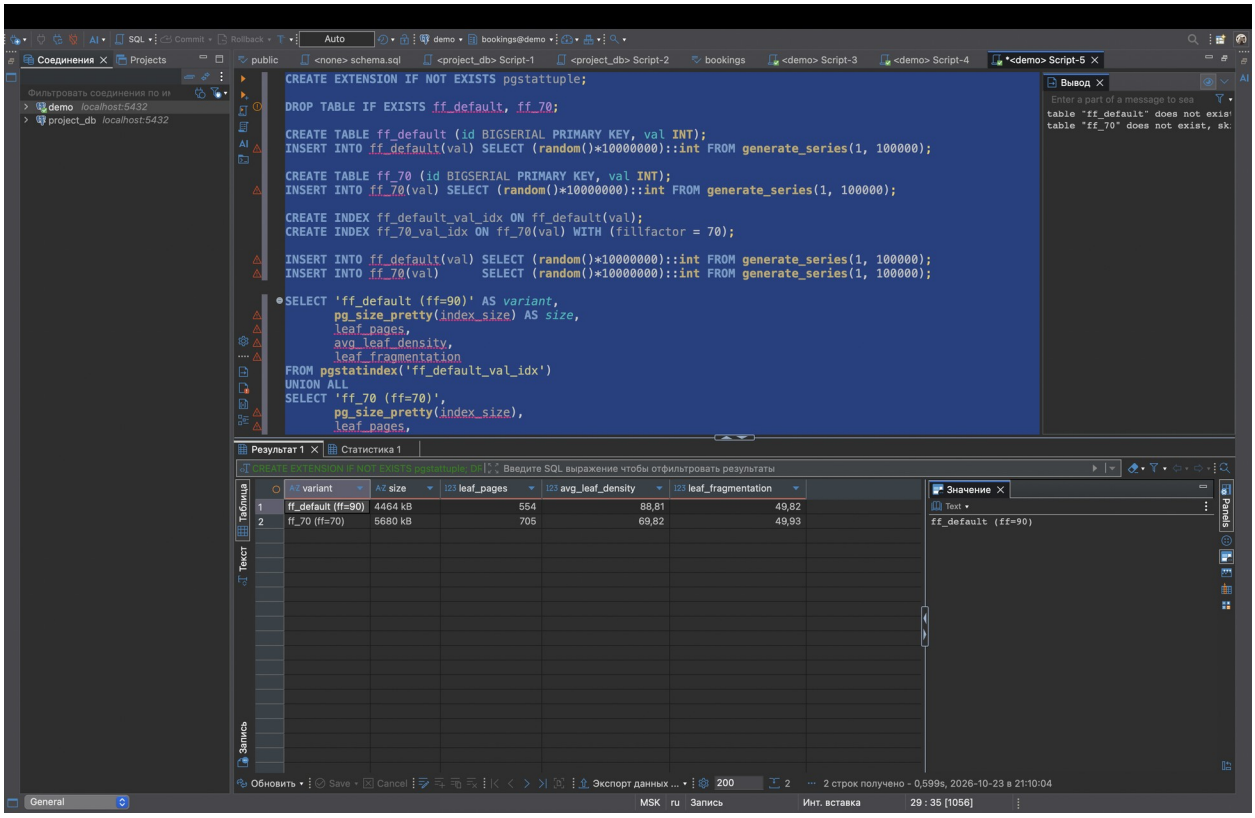
Количество строк в таблицах. Число строк в hot_serial и hot_uuid практически совпадает с числом транзакций, обработанных pgbench в соответствующем прогоне, то есть каждая транзакция добавила ровно одну строку и ни одна вставка не потерялась. Небольшая разница между таблицами объясняется тем, что pgbench работал фиксированное время

(20 секунд), а не фиксированное число транзакций. Поэтому сравнивать надо индексы по размеру на строку, а не по абсолютным значениям.

Размеры индексов первичных ключей. Индекс hot_uuid_pkey почти вдвое больше hot_serial_pkey при том же числе строк. Во-первых, UUID занимает 16 байт против 8 у BIGINT, поэтому на странице помещается меньше записей. Во-вторых, при монотонном BIGSERIAL новые значения всегда попадают в правый край индекса, и страницы заполняются почти до конца, а случайные UUID вставляются в произвольные листья, вызывают page splits, и страницы остаются заполнены неплотно. Так подтверждается, что снятие hot spot оплачивается размером индекса и худшей локальностью кэш

Раздел 7. Практикум 5: fillfactor и Page Splits



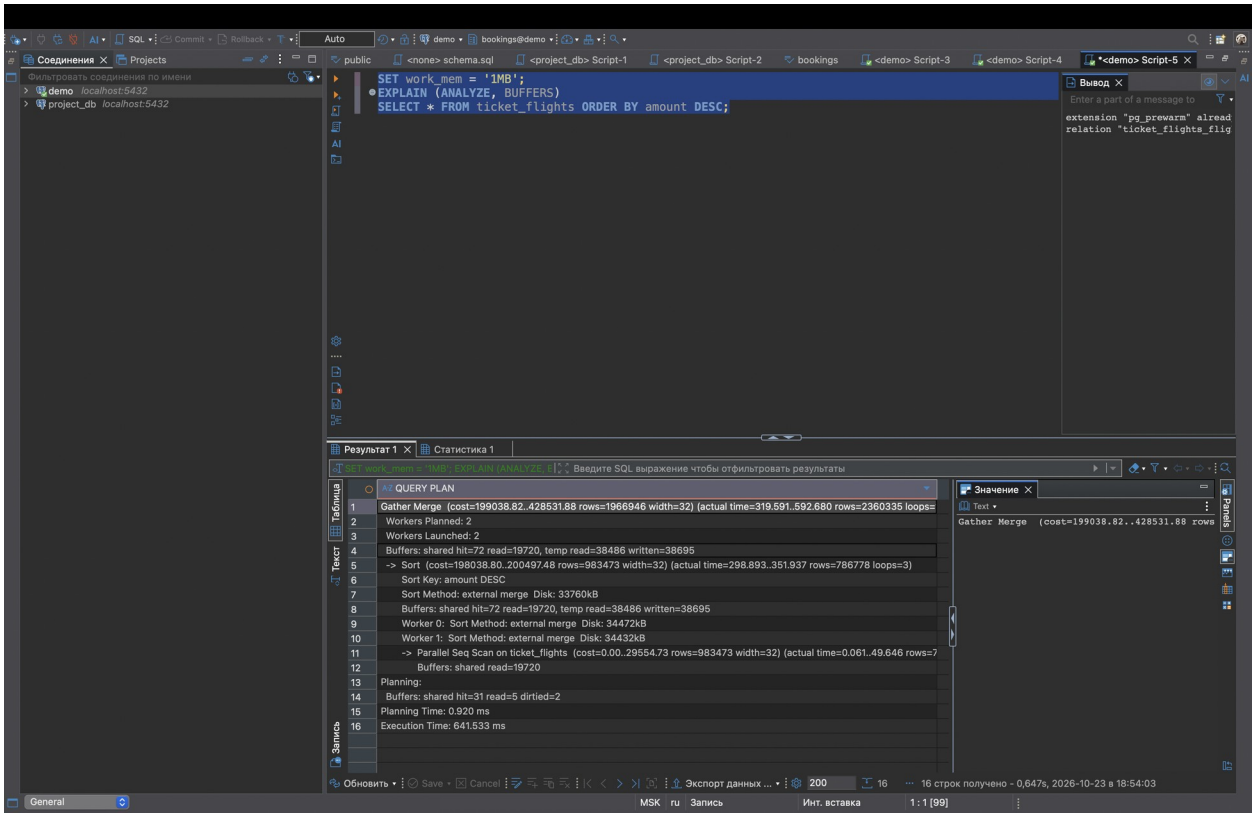


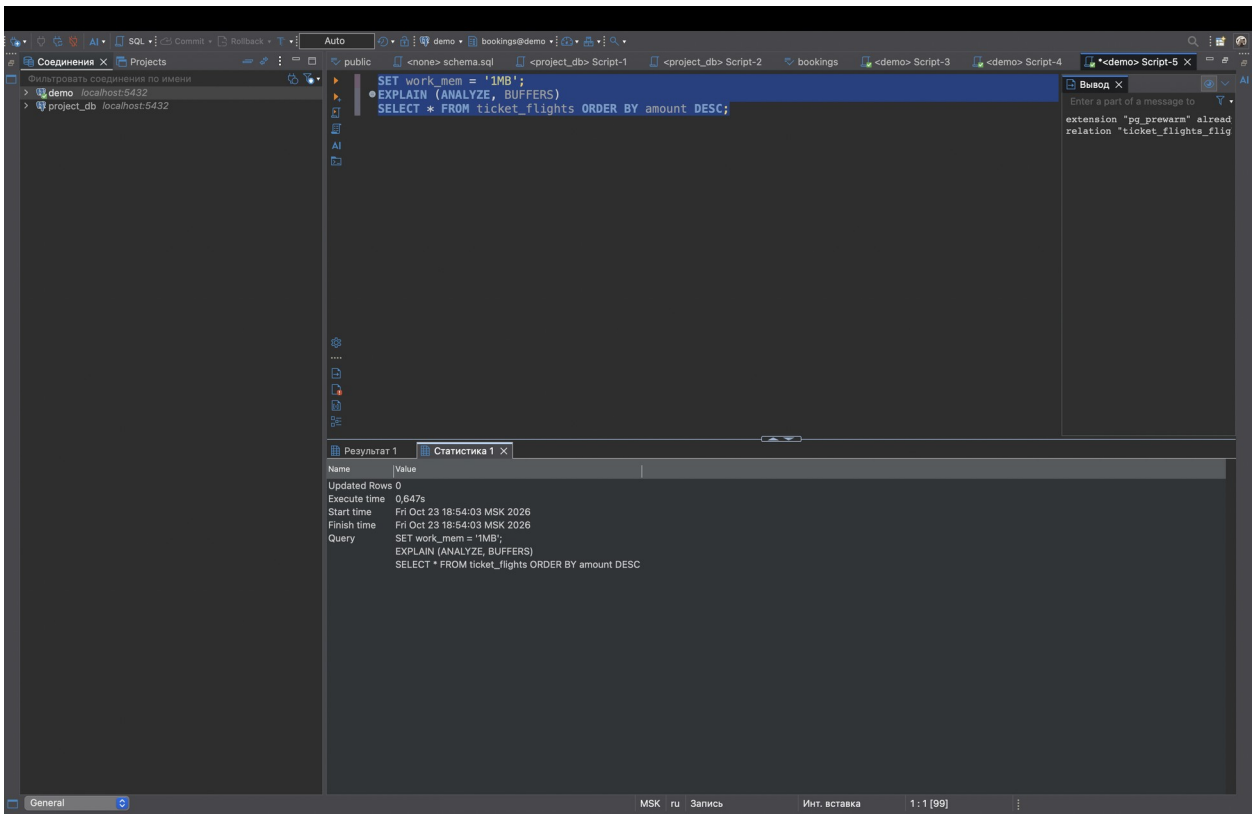
Влияние fillfactor на индексы. avg_leaf_density показывает, какая доля места в листовых страницах занята данными, а leaf_fragmentation показывает долю листовых страниц, физический порядок которых не совпадает с логическим (это влияет на скорость сканирования диапазонов). Индекс с fillfactor = 70 оставил на страницах резерв, поэтому

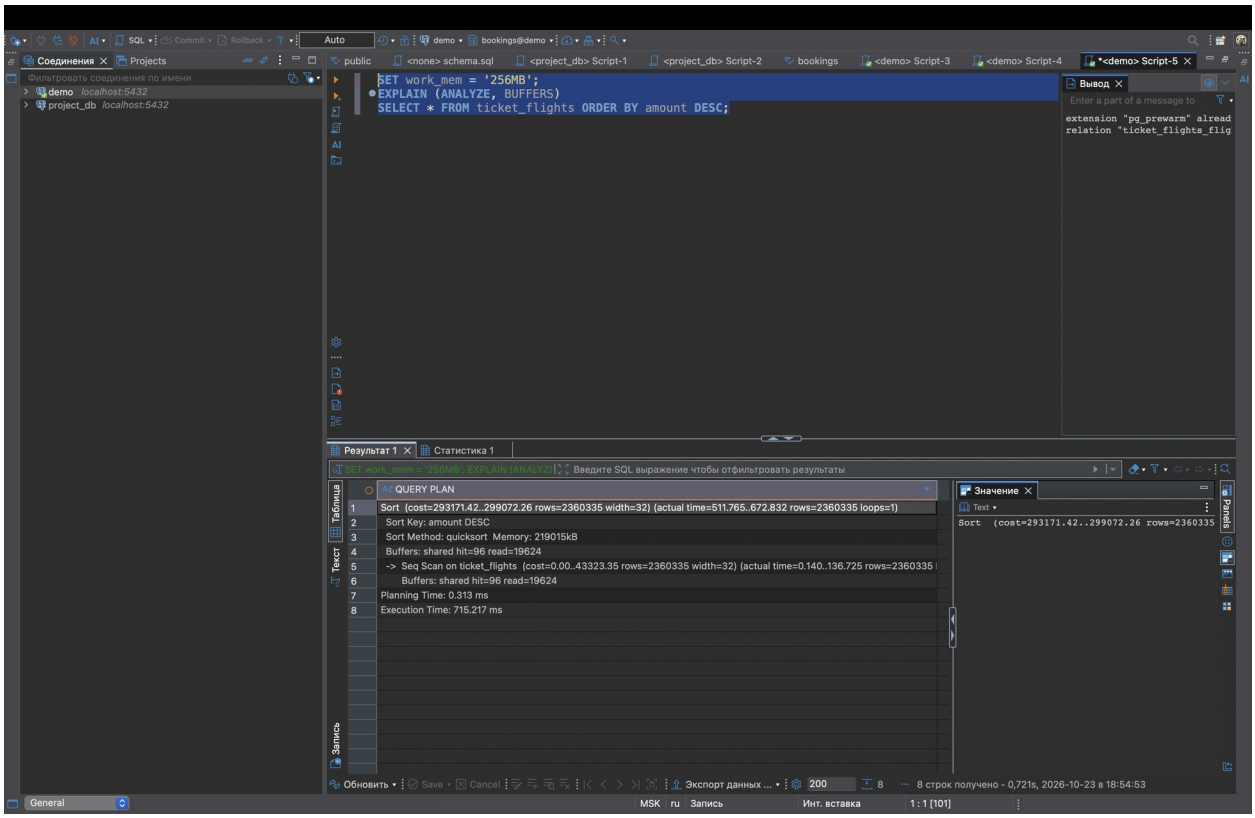
менее плотный при 70 (около 70% против 89%), но крупнее: плотность ниже, размер больше примерно на четверть. Фрагментация при этом почти не изменилась: после удвоения числа строк случайными значениями резерв в 30% всё равно закончился, и страницы расщеплялись в обоих индексах. Низкий fillfactor выигрывает, когда вставок немного относительно резерва, и обходится ценой места. Его имеет смысл задавать для индексов с частыми UPDATEиндексируемых столбцов и для индексов со случайными ключами (например, UUID), которые регулярно дополняются

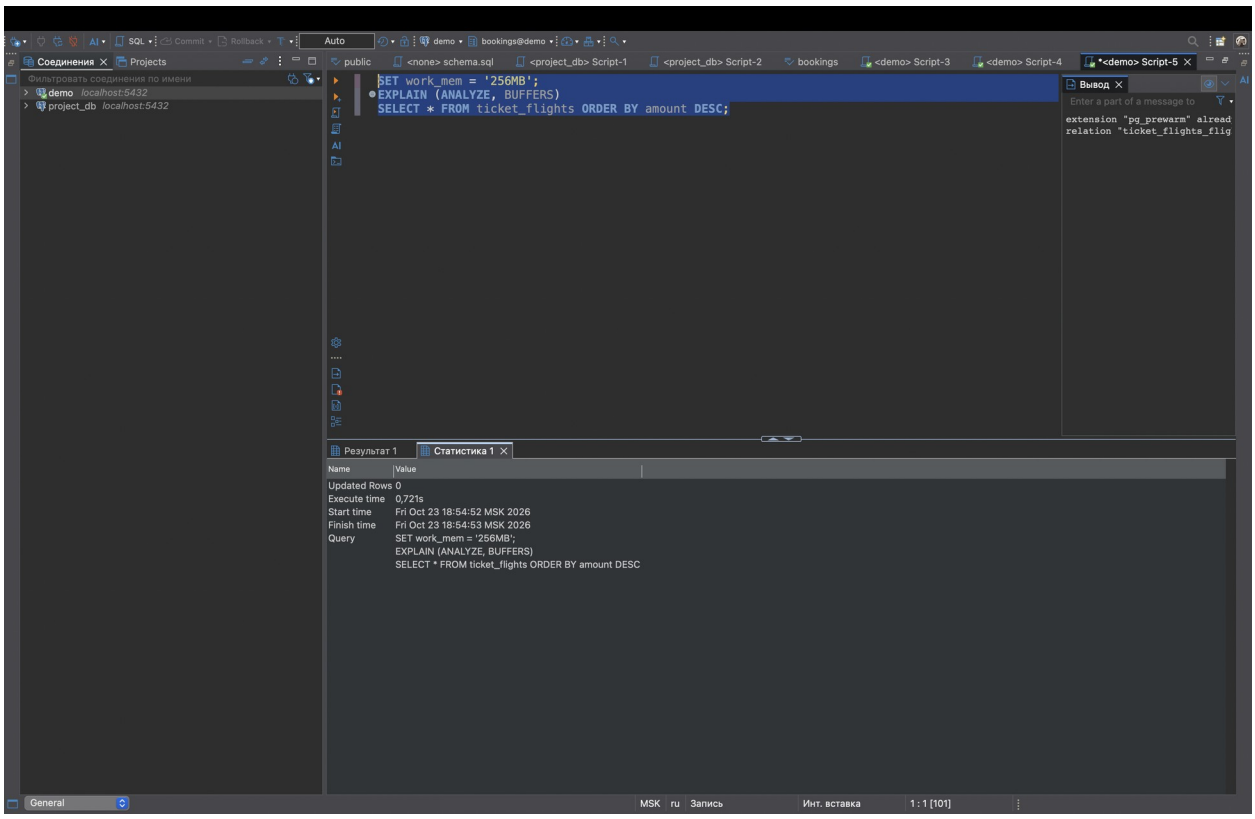
Раздел 8. Практикум 6: Wait Events Forensics

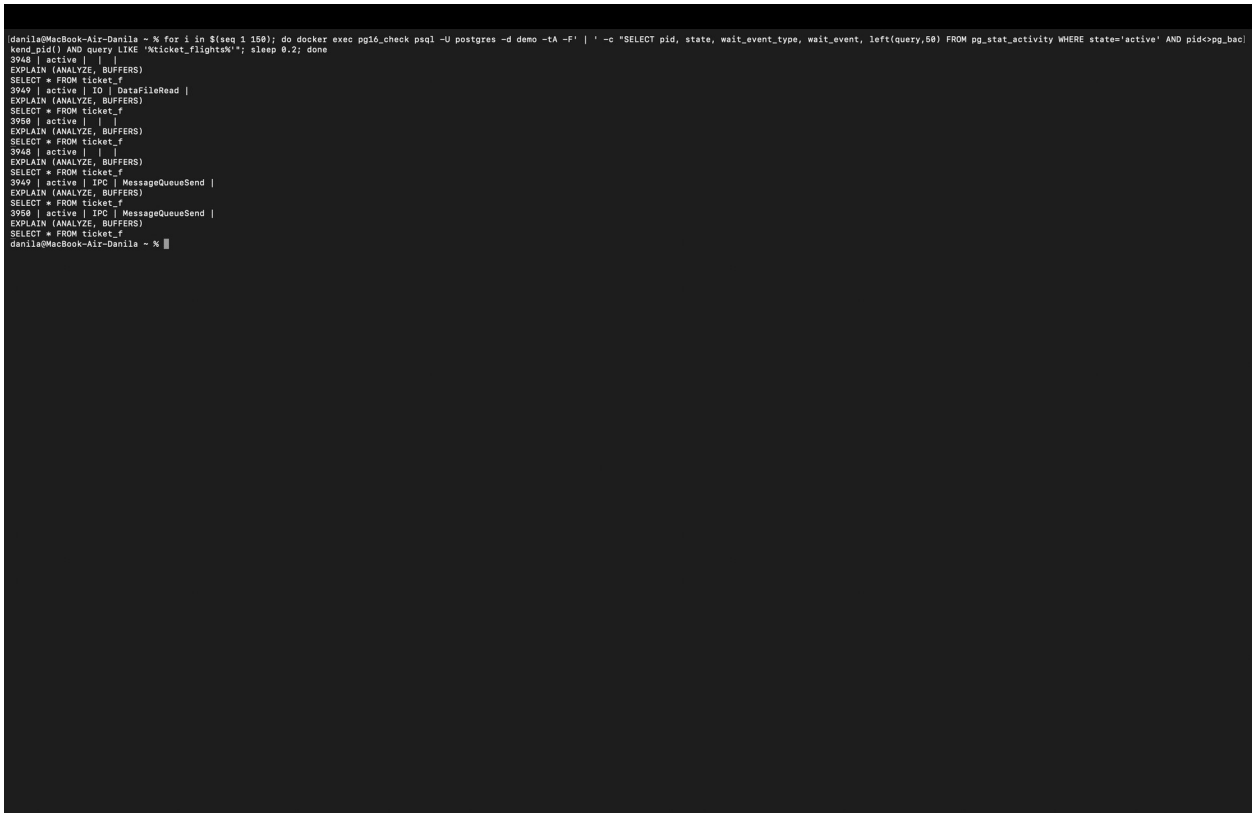
Сессия 1

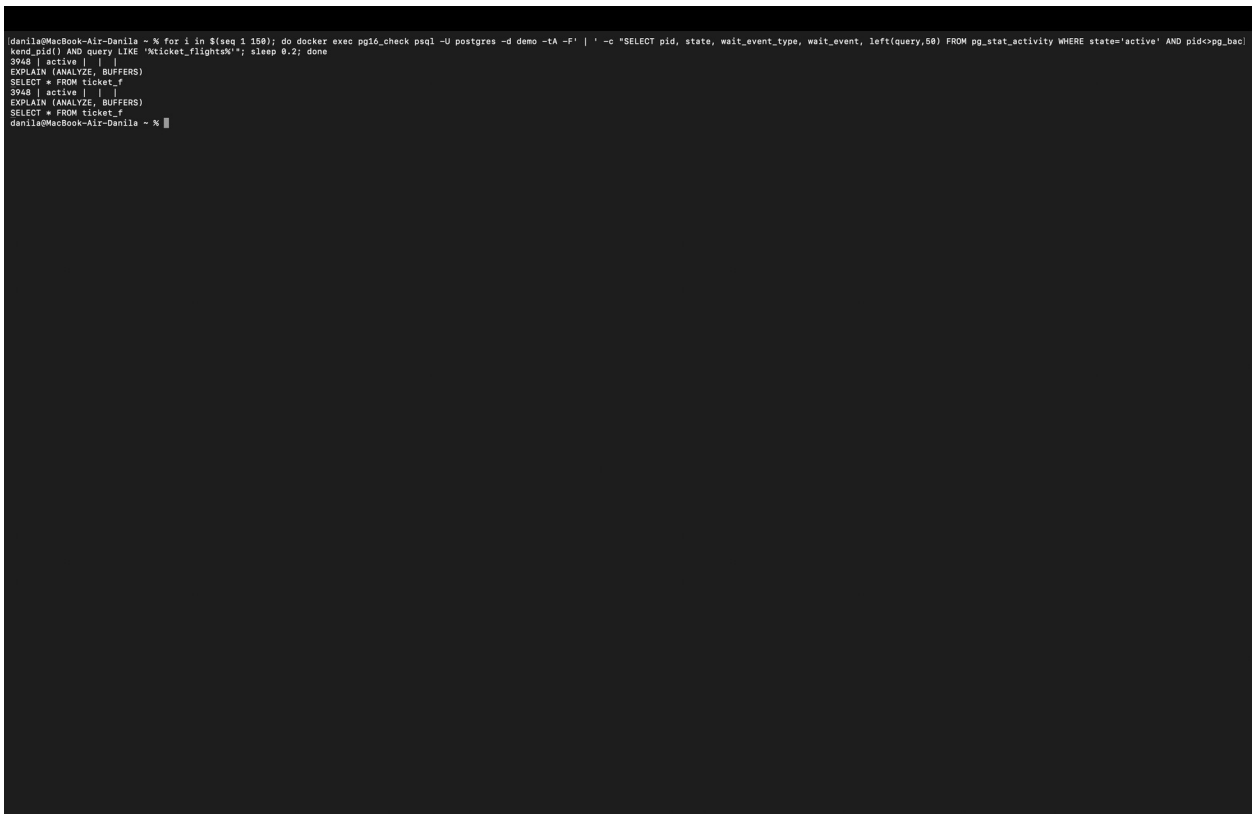



SET work_mem = '1MB';
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_flights ORDER BY amount DESC;
Результат 1
Статистика 1
Name
Value
Updated Rows 0
Execute time 0,647s
Start time Fri Oct 23 18:54:03 MSK 2026
Finish time Fri Oct 23 18:54:03 MSK 2026
Query SET work_mem = '1MB';
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_flights ORDER BY amount DESC


SET work_mem = '256MB';
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_flights ORDER BY amount DESC;
Результат 1
Статистика 1
QUERY PLAN
1 Sort (cost=293171.42..299072.26 rows=2360335 width=32) (actual time=511.765..672.832 rows=2360335 loops=1)
2 Sort Key: amount DESC
3 Sort Method: quicksort Memory: 219015kB
4 Buffers: shared hit=96 read=19624
5 -> Seq Scan on ticket_flights (cost=0.00..43323.35 rows=2360335 width=32) (actual time=0.140..136.725 rows=2360335
6 Buffers: shared hit=96 read=19624
7 Planning Time: 0.313 ms
8 Execution Time: 715.217 ms
Значение
Sort (cost=293171.42..299072.26 rows=2360335
8 строк получено - 0,721s, 2026-10-23 в 18:54:53


SET work_mem = '256MB';
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_flights ORDER BY amount DESC;
Результат 1
Статистика 1
Name
Value
Updated Rows 0
Execute time 0,721s
Start time Fri Oct 23 18:54:52 MSK 2026
Finish time Fri Oct 23 18:54:53 MSK 2026
Query SET work_mem = '256MB';
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_flights ORDER BY amount DESC

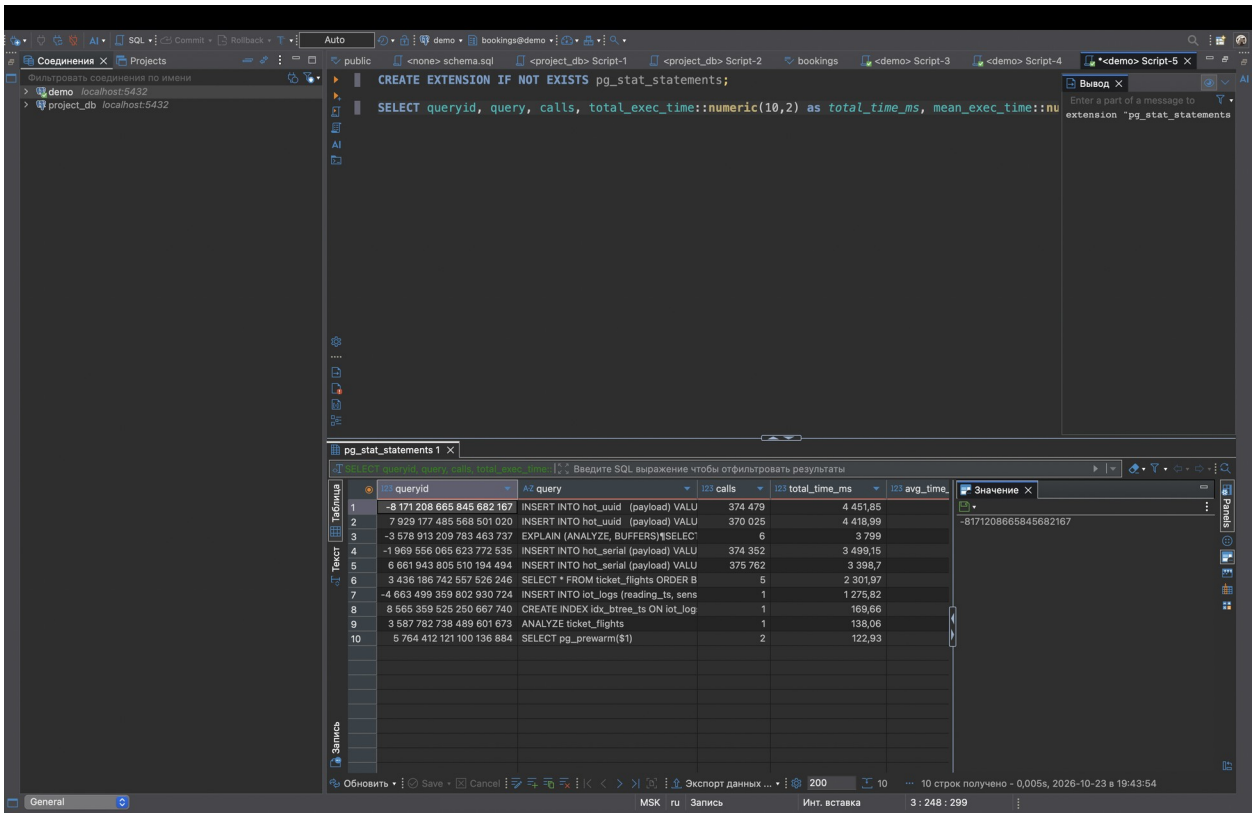

danila@MacBook-Air-Danila ~ % for i in $(seq 1 150); do docker exec pg16_check psql -U postgres -d demo -tA -F' | ' -c "SELECT pid, state, wait_event_type, wait_event, left(query,50) FROM pg_stat_activity WHERE state='active' AND pid<>pg_backend_pid() AND query LIKE '%ticket_flights%'"; sleep 0.2; done
3948 | active | | |
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_f
3949 | active | IO | DataFileRead |
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_f
3950 | active | | |
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_f
3948 | active | | |
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_f
3949 | active | IPC | MessageQueueSend |
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_f
3950 | active | IPC | MessageQueueSend |
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ticket_f
danila@MacBook-Air-Danila ~ %

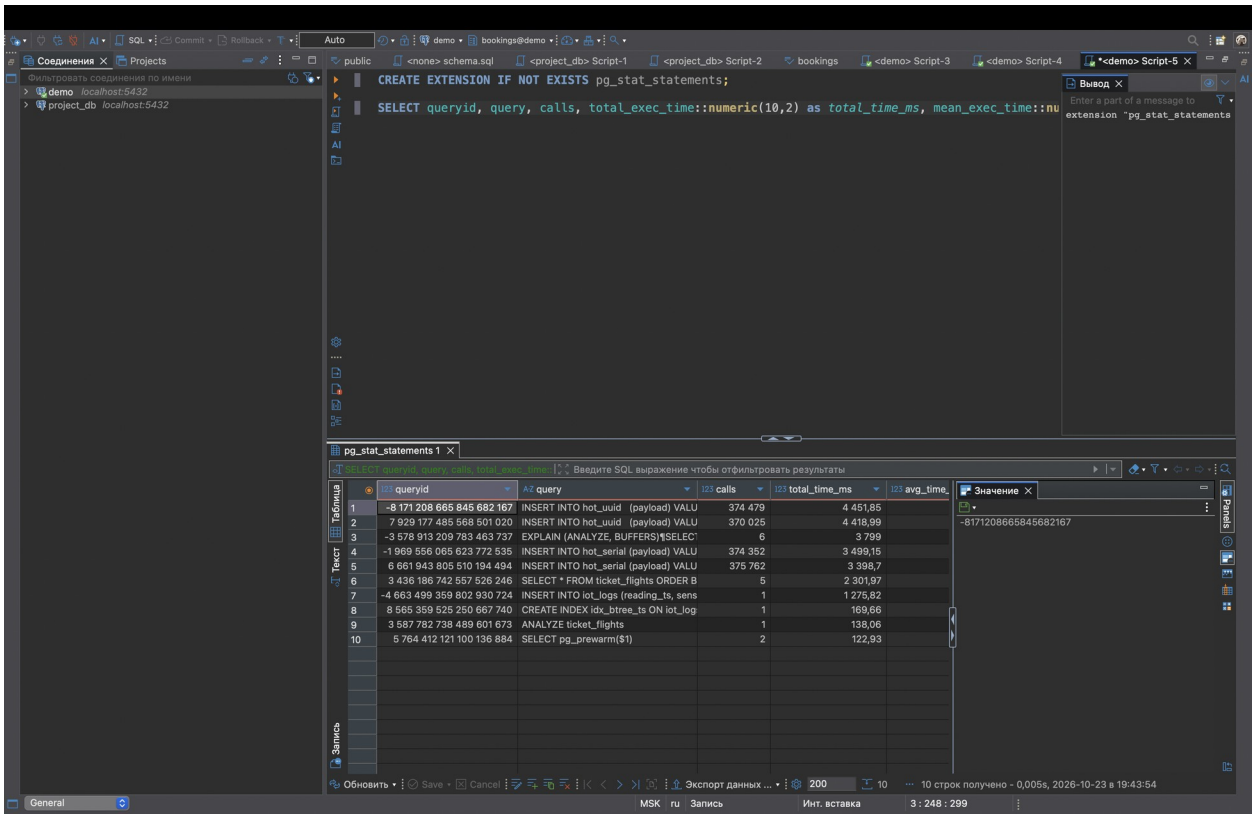


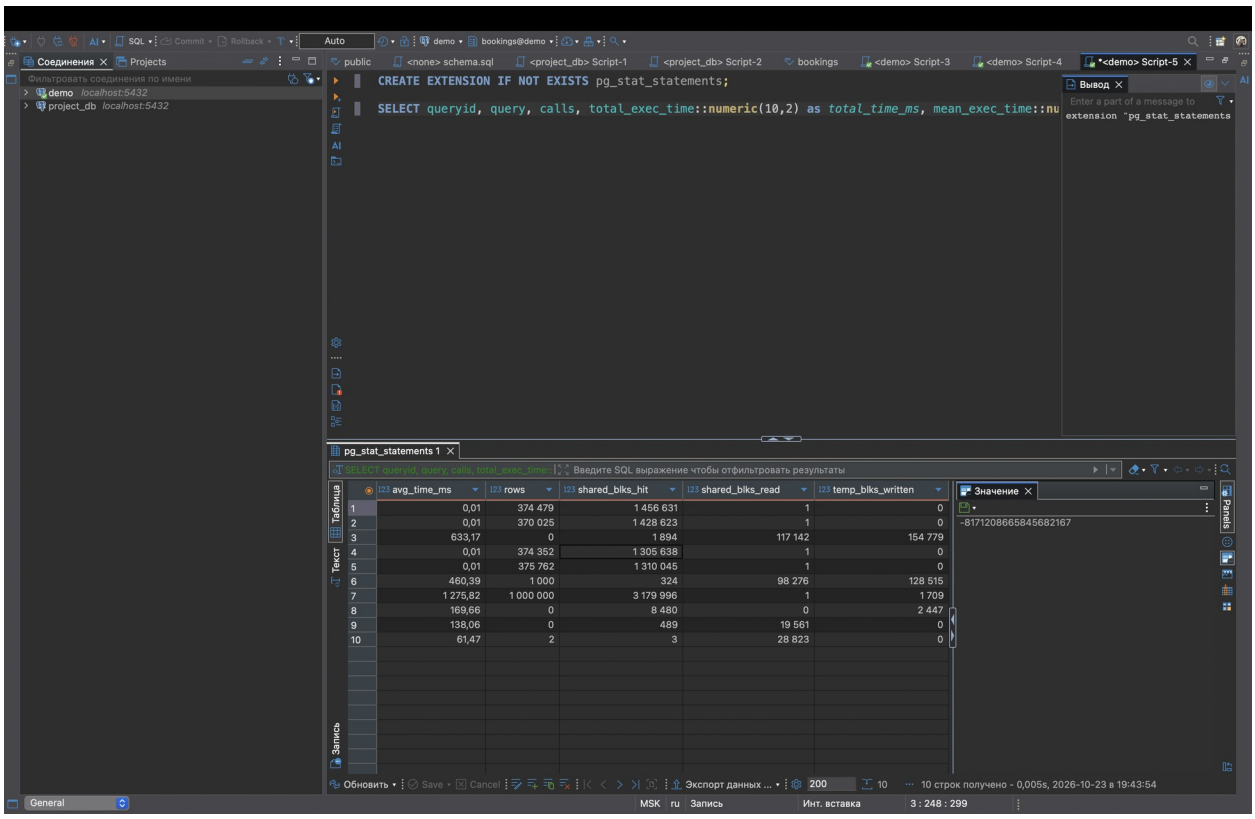
Замена \watch. Задание предлагает запрос с \watch 1, но это метакоманда psql, и DBeaver её не поддерживает. Кроме того, сортировка выполняется около 0,7 с, поэтому опрос раз в секунду почти ничего бы не поймал. Поэтому наблюдение за ожиданиями выполнено в терминале: цикл из 150 итераций с паузой 0,2 с через docker exec и psql выполняет тот же запрос к pg_stat_activity с фильтром по тексту запроса, пока в другом окне идёт EXPLAIN (ANALYZE, BUFFERS). Результат тот же, что у \watch, только выборка чаще.

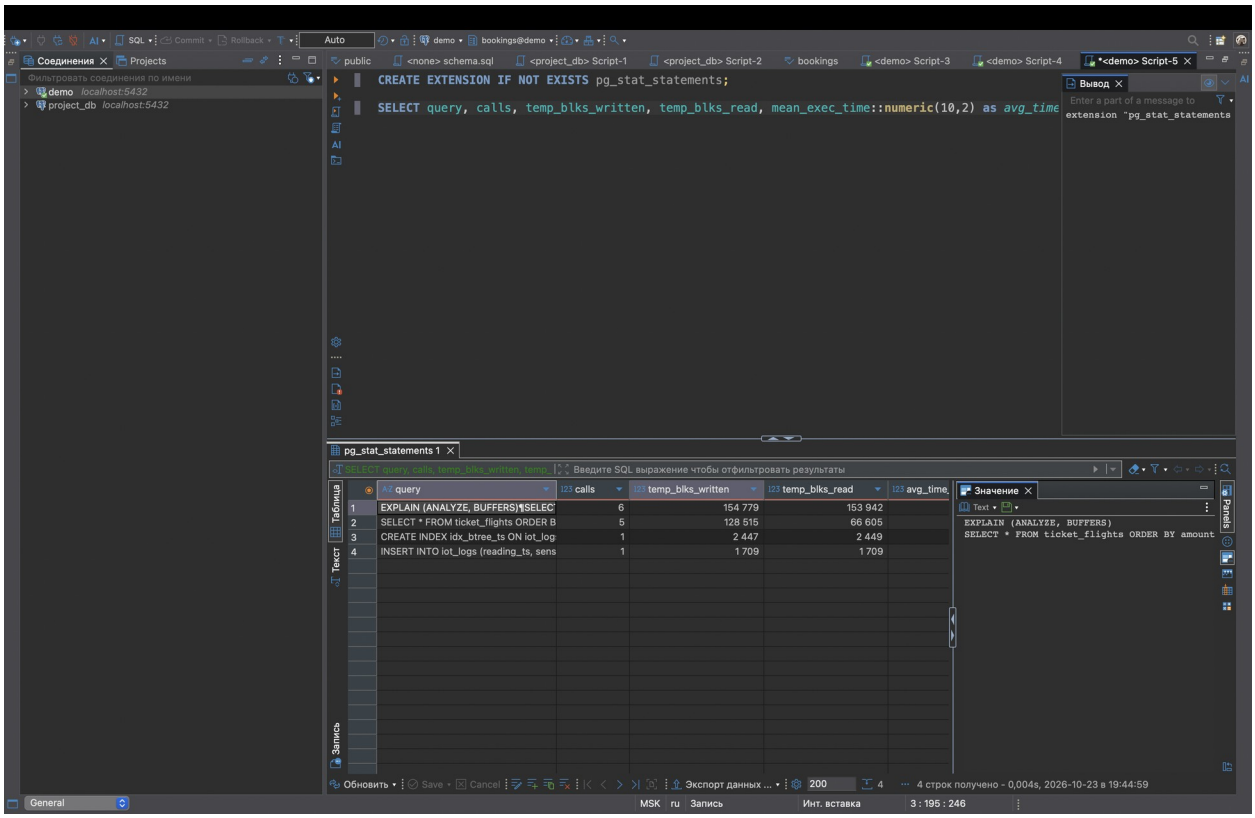
События ожидания и методы сортировки. В выборке при work_mem = 1MB (external merge) пойманы IO:DataFileRead (чтение страниц таблицы)
и IPC:MessageQueueSend (передача данных от параллельных воркеров ведущему процессу в Gather Merge), остальные снимки пустые. Событий записи и чтения временных файлов (BufferFileWrite, BufferFileRead) выборка не зафиксировала: сортировка занимает доли секунды, а опрос идёт раз в 0,2 с. Про сброс на диск надёжнее говорят Disk и temp read/written в плане. При 256MB (quicksort) в выборке только пустые события: запрос работает на процессоре, воркеров и обмена через очередь нет, временных файлов нет. Выборка небольшая, поэтому вывод осторожный: сортировка в памяти не оставляет событий ввода-вывода, связанных с временными файлами.

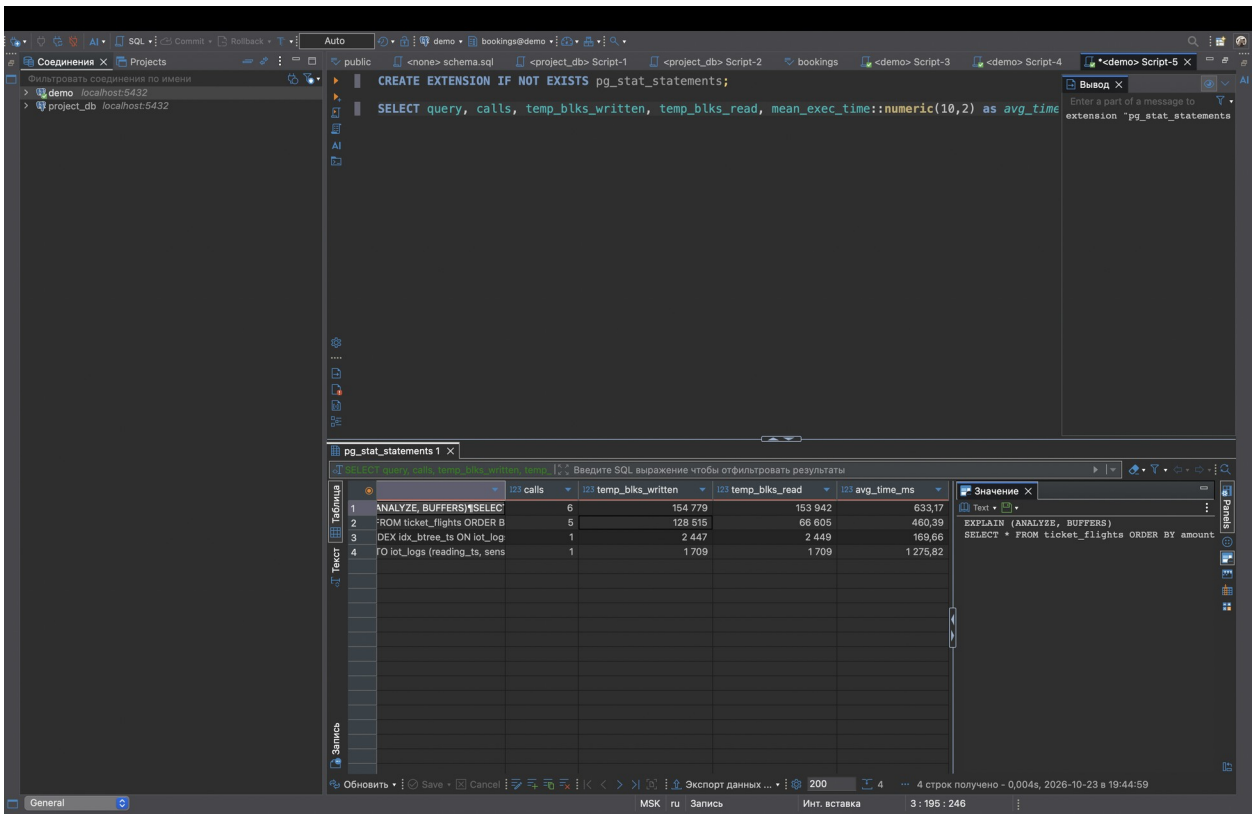
Раздел 9. Диагностика через pg_stat_statements

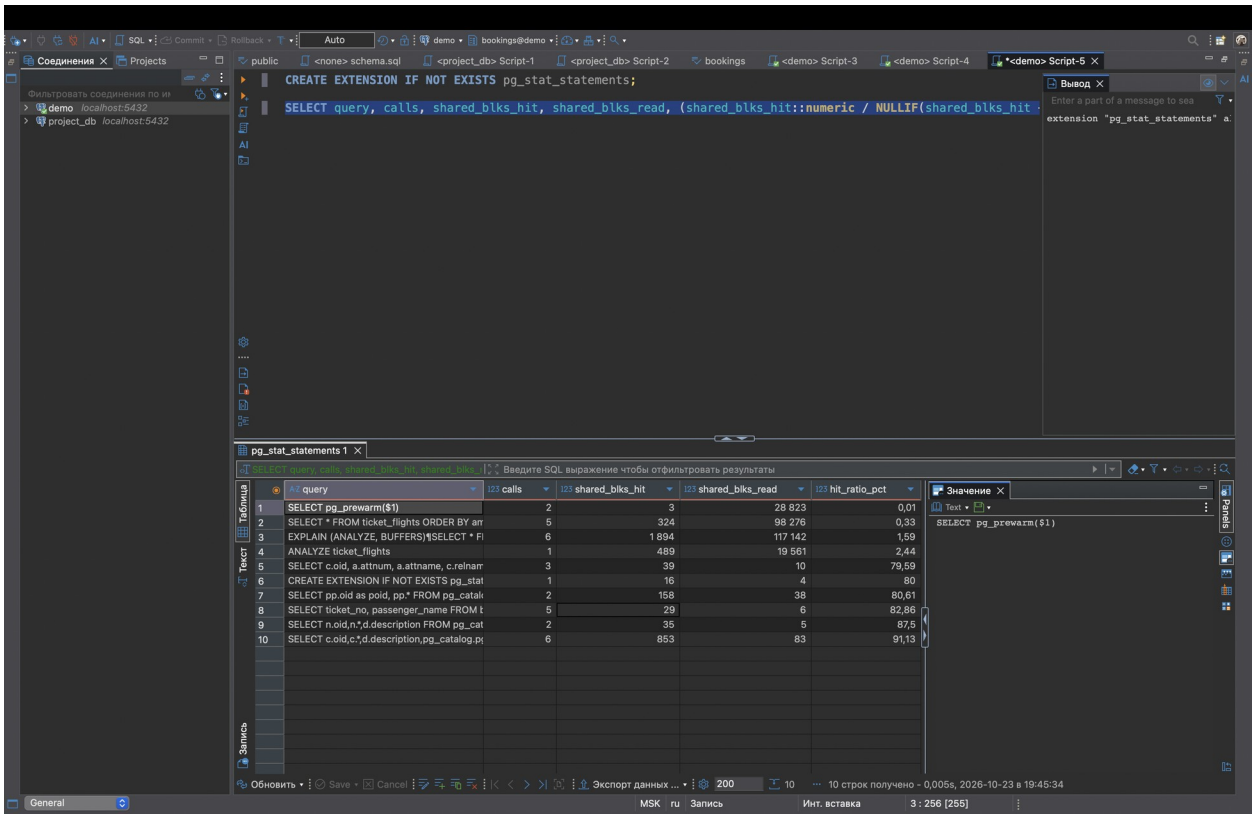





CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SELECT queryid, query, calls, total_exec_time::numeric(10,2) as total_time_ms, mean_exec_time::nu
pg_stat_statements 1
avg_time_ms rows shared_blks_hit shared_blks_read temp_blks_written
0,01 374 479 1 456 631 1 0
0,01 370 025 1 428 623 1 0
633,17 0 1 894 117 142 154 779
0,01 374 352 1 305 638 1 0
0,01 375 762 1 310 045 1 0
460,39 1 000 324 98 276 128 515
1 275,82 1 000 000 3 179 996 1 1 709
169,66 0 8 480 0 2 447
138,06 0 489 19 561 0
61,47 2 3 28 823 0
10 строк получено - 0,005s, 2026-10-23 в 19:43:54


CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SELECT query, calls, temp_blks_written, temp_blks_read, mean_exec_time::numeric(10,2) as avg_time
pg_stat_statements 1
query calls temp_blks_written temp_blks_read avg_time
EXPLAIN (ANALYZE, BUFFERS)¶SELEC 6 154 779 153 942
SELECT * FROM ticket_flights ORDER B 5 128 515 66 605
CREATE INDEX idx_btree_ts ON iot_log 1 2 447 2 449
INSERT INTO iot_logs (reading_ts, sens 1 1 709 1 709
4 строк получено - 0,004s, 2026-10-23 в 19:44:59





Диагностика pg_stat_statements. В топе по общему времени оказались вставки hot_uuid и hot_serial из pgbench, но это не медленные запросы: средняя длительность 0,01 мс, наверху они из-за сотен тысяч вызовов. По-настоящему тяжёлые здесь аналитические запросы: SELECT * FROM ticket_flights ORDER BY amount DESC (в среднем 460 мс) и его EXPLAIN (ANALYZE, BUFFERS) (около 633 мс), а также разовая INSERT INTO iot_logs на 1 млн строк (около 1,3 с). В список со сбросом на диск попали четыре запроса: оба варианта сортировки по amount(больше всего temp_blks_written, потому что

сортировка 2,36 млн строк не помещается в work_mem), CREATE INDEX idx_btree_ts и INSERT INTO iot_logs. Для CREATE INDEX временные файлы объясняются нехваткой maintenance_work_mem, а не work_mem. В топе с низким cache hit ratio оказались pg_prewarm (0,01%, он читает таблицу с диска по своему назначению), сортировка ticket_flights (0,33%: таблица около 154 МБ не помещается в shared_buffers 128 МБ и читается заново при каждом вызове), EXPLAIN этого запроса и ANALYZE ticket_flights. Вывод: на стенде главный источник неэффективности один, полное чтение и сортировка ticket_flights без ограничения выборки. Его лечат увеличением work_mem, выбором только нужных столбцов с LIMIT или индексом по amount. Частые вставки на этом фоне заметны только числом вызовов и сами проблемой не являются.

7. Теоретическое задание: назначение системных представлений статистики

Таблица 1. pg_stat_user_tables

Характеристика	**Описание**
1. Тип и схема	Системное представление в pg_catalog.
2. Ключевые столбцы	last_analyze, last_autoanalyze (время последнего сбора статистики); n_mod_since_analyze (число измененных строк); seq_scan, idx_scan (счетчики сканирований); n_live_tup, n_dead_tup (живые и мертвые строки).
3. Значение столбцов	Временные метки (абсолютные значения), счетчики (накопительные), количество строк (оценочные значения планировщика).
4. Назначение	Поиск таблиц с устаревшей статистикой, контроль работы автовакуума, проверка необходимости ручного ANALYZE, оценка раздувания (bloat).
5. Пример из ДЗ	Проверка таблицы flights перед выполнением запросов: метрики last_analyze были NULL, а после ручного ANALYZE flights; обновились.
6. Ограничения	Значения n_live_tup и n_dead_tup являются приблизительными оценками (статистикой), а не точными транзакционными данными.

Таблица 2. pg_stat_activity

Характеристика	**Описание**
1. Тип и схема	Системное представление в pg_catalog.
2. Ключевые столбцы	state (статус сессии); wait_event_type, wait_event (тип и причина ожидания); query (текст запроса); query_start (время запуска); backend_type (тип процесса).

3. Значение столбцов	Текстовые статусы, абсолютные временные метки. Представляет собой моментальный срез (snapshot) текущего состояния, а не историю.
4. Назначение	Мониторинг базы данных в реальном времени, поиск зависших или долгих транзакций, выявление блокировок и узких мест (I/O, CPU, Locks).
5. Пример из ДЗ	В цикле терминала отслеживались wait_event для запроса сортировки ticket_flights при work_mem = 1MB (наблюдалось I/O) и 256MB (событий ожидания не было).
6. Ограничения	Не показывает короткие запросы, которые успели выполниться между обновлениями представления. Текст запроса в query по умолчанию обрезается (обычно до 1024 символов).

Таблица 3. pg_class

Характеристика	**Описание**
1. Тип и схема	Системная таблица каталога в pg_catalog.
2. Ключевые столбцы	relname (имя объекта); relkind (тип: r — таблица, i — индекс); relpages (количество страниц); reltuples (количество строк). Функции: pg_relation_size(), pg_total_relation_size().
3. Значение столбцов	Системные идентификаторы, символьные флаги, количество дисковых блоков (обычно по 8 КБ), байты (для функций размера).
4. Назначение	Получение физических характеристик объектов БД, определение размеров таблиц и индексов, проверка структуры базы.
5. Пример из ДЗ	Сравнение физического размера индексов B-Tree и BRIN для таблицы test_skew с помощью вызова pg_relation_size(oid).
6. Ограничения	Столбцы relpages и reltuples обновляются только при выполнении VACUUM или ANALYZE, поэтому могут содержать сильно устаревшие данные.

Таблица 4. pg_stat_database

Характеристика	**Описание**
1. Тип и схема	Системное представление в pg_catalog.
2. Ключевые столбцы	blks_hit, blks_read (обращения к кэшу и диску); tup_returned, tup_fetched (прочитанные строки); xact_commit,

	xact_rollback (успешные/отмененные транзакции).
3. Значение столбцов	Накопительные счетчики с момента создания базы данных или последнего сброса статистики.
4. Назначение	Оценка общего состояния БД, вычисление глобального Cache Hit Ratio, выявление аномальных всплесков откатов транзакций (rollbacks).
5. Пример из ДЗ	Общий анализ интенсивности чтения с диска в БД demo для оценки необходимости прогрева кэша.
6. Ограничения	Дает только среднюю картину по всей базе данных (слишком высокий уровень агрегации). Не позволяет найти конкретный проблемный запрос или таблицу.

Таблица 5. pg_stat_statements

Характеристика	**Описание**
1. Тип и схема	Расширение (extension) и одноименное представление.
2. Ключевые столбцы	queryid (хэш запроса); calls (число вызовов); total_exec_time, mean_exec_time (время выполнения); shared_blks_hit, shared_blks_read (кэш/диск); temp_blks_written (сброс на диск); rows.
3. Значение столбцов	Накопительные значения, миллисекунды для времени, количество блоков (по 8 КБ) для данных.
4. Назначение	Профилирование производительности, поиск самых долгих, ресурсоемких или часто вызываемых запросов, выявление запросов, которым не хватает work_mem.
5. Пример из ДЗ	Поиск топ-10 самых медленных запросов и запросов с temp_blks_written > 0 после сброса статистики.
6. Ограничения	Требует добавления в shared_preload_libraries (нужен рестарт СУБД). Нормализует литералы, скрывая конкретные параметры проблемных запросов.

Таблица 6. pg_prewarm

Характеристика	**Описание**
1. Тип и схема	Расширение (extension) и функция.
2. Ключевые столбцы	Функция pg_prewarm() принимает имя таблицы/индекса, тип форка (main) и диапазон блоков.

3. Значение столбцов	Возвращает количество успешно загруженных в кэш блоков.
4. Назначение	Принудительный прогрев кэша (ОС или shared_buffers) перед запуском тяжелой аналитики или после рестарта СУБД, чтобы избежать задержек чтения с диска (cold cache).
5. Пример из ДЗ	Вызов функции для ticket_flights и ticket_flights_flight_id_amount_idx, что привело к снижению метрики read и росту hit в EXPLAIN.
6. Ограничения	Если размер таблицы превышает объем shared_buffers, функция вытеснит другие полезные данные, а часть прогретой таблицы может быть сразу же вытеснена алгоритмом LRU/Clock.

Таблица 7. pgstattuple / pgstatindex

Характеристика	**Описание**
1. Тип и схема	Расширения (extensions) и функции профилирования.
2. Ключевые столбцы	avg_leaf_density (плотность заполнения листьев); leaf_fragmentation (уровень фрагментации); leaf_pages (количество листьев); index_size; free_percent.
3. Значение столбцов	Проценты (от 0 до 100), байты, количество дисковых страниц.
4. Назначение	Точный анализ раздувания (bloat) индексов и таблиц. Оценка влияния параметра fillfactor на расщепление страниц (page splits).
5. Пример из ДЗ	Сравнение индексов ff_default_val_idx и ff_70_val_idx, чтобы показать, как fillfactor = 70 резервирует место и снижает плотность заполнения листьев.
6. Ограничения	Функции выполняют полное сканирование индексов/таблиц. На больших базах это создает огромную I/O-нагрузку и может занять продолжительное время.

Таблица 8. pg_stat_bgwriter (или pg_stat_io)

Характеристика	**Описание**
1. Тип и схема	Системное представление в pg_catalog.
2. Ключевые столбцы	checkpoints_timed (по расписанию); checkpoints_req (по объему WAL); buffers_checkpoint, buffers_backend (сброшено блоков фоновыми процессами и самими клиентами); buffers_alloc.
3. Значение столбцов	Накопительные глобальные счетчики для

	всего кластера с момента последнего сброса статистики.
4. Назначение	Настройка параметров контрольных точек (checkpoint_timeout, max_wal_size), диагностика I/O. Если buffers_backend быстро растет, значит рабочим процессам не хватает фоновой записи (нужен тюнинг bgwriter).
5. Пример из ДЗ	Оценка общей картины дискового ввода-вывода кластера и баланса между плановыми и экстренными чекпоинтами.
6. Ограничения	Абсолютно глобальная метрика. По ней невозможно понять, какая конкретно таблица или запрос вызвали аномальную активность записи.

Аналитический вывод

В ходе выполнения практической работы наиболее полезными инструментами оказались pg_stat_statements и pg_stat_activity. pg_stat_statements выступает лучшим инструментом для ретроспективного анализа, позволяя точно находить самые ресурсоемкие запросы, оценивать использование диска (temp_blks_written) и попадание в кэш. pg_stat_activity незаменим для онлайн-мониторинга: именно с его помощью удалось в реальном времени зафиксировать ожидание I/O во время медленной сортировки слиянием (external merge).

При следующей диагностике аналогичного стенда с проблемами производительности в первую очередь следует проверять:

- pg_stat_activity — для выявления текущих блокировок и событий ожидания (wait events), чтобы понять, простаивает ли база на дисковом вводе-выводе или страдает от lock contention.
- pg_stat_statements — для определения конкретных SQL-запросов, потребляющих больше всего времени и буферов (выявление плохих планов выполнения).
- pg_stat_user_tables — для проверки актуальности статистики. Устаревшая статистика (null или большой n_mod_since_analyze) — главная причина того, что планировщик строит неоптимальные маршруты для тяжелых аналитических запросов.